

C++ Bayesian Filtering Library

Generated by Doxygen 1.9.8

1 About this library	1
1.1 Dependencies	1
1.1.1 EIGEN	1
1.1.2 Doxygen (optional)	1
1.2 Demos	2
2 Namespace Index	5
2.1 Namespace List	5
3 Hierarchical Index	7
3.1 Class Hierarchy	7
4 Class Index	9
4.1 Class List	9
5 File Index	11
5.1 File List	11
6 Namespace Documentation	13
6.1 BayesianFiltering Namespace Reference	13
6.1.1 Typedef Documentation	14
6.1.1.1 Result	14
6.1.2 Function Documentation	14
6.1.2.1 MultivariateGaussianSampler()	14
6.1.2.2 MultivariateZeroMeanGaussianSampler()	15
6.1.2.3 ResultFormat() [1/2]	15
6.1.2.4 ResultFormat() [2/2]	15
6.1.2.5 UnivariateGaussianSampler()	16
6.1.2.6 UnivariateUniformSampler()	16
7 Class Documentation	17
7.1 BayesianFiltering::AdaptiveKalmanFilter Class Reference	17
7.1.1 Detailed Description	19
7.1.2 Constructor & Destructor Documentation	20
7.1.2.1 ~AdaptiveKalmanFilter()	20
7.1.3 Member Function Documentation	20
7.1.3.1 Correction()	20
7.1.3.2 Prediction()	20
7.1.3.3 Reset() [1/2]	20
7.1.3.4 Reset() [2/2]	21
7.1.4 Member Data Documentation	21
7.1.4.1 alpha	21
7.1.4.2 beta	21
7.2 BayesianFiltering::BayesianFilter Class Reference	22

7.2.1 Detailed Description	23
7.2.2 Constructor & Destructor Documentation	23
7.2.2.1 BayesianFilter()	23
7.2.2.2 ~BayesianFilter()	23
7.2.3 Member Function Documentation	23
7.2.3.1 Reset()	23
7.2.4 Member Data Documentation	24
7.2.4.1 d2_corrected	24
7.2.4.2 d2_predicted	24
7.2.4.3 P	24
7.2.4.4 S_corrected	24
7.2.4.5 S_predicted	24
7.2.4.6 V_corrected	24
7.2.4.7 V_predicted	25
7.2.4.8 X	25
7.2.4.9 Y_corrected	25
7.2.4.10 Y_predicted	25
7.3 BayesianFiltering::ExtendedKalmanFilter Class Reference	25
7.3.1 Detailed Description	27
7.3.2 Constructor & Destructor Documentation	27
7.3.2.1 ~ExtendedKalmanFilter()	27
7.3.3 Member Function Documentation	27
7.3.3.1 Correction()	27
7.3.3.2 Prediction()	28
7.3.3.3 Reset()	28
7.4 BayesianFiltering::KalmanFilter Class Reference	29
7.4.1 Detailed Description	31
7.4.2 Constructor & Destructor Documentation	31
7.4.2.1 ~KalmanFilter()	31
7.4.3 Member Function Documentation	31
7.4.3.1 Correction()	31
7.4.3.2 Prediction()	31
7.4.3.3 Reset()	32
7.5 BayesianFiltering::LinearBayesianFilter Class Reference	32
7.5.1 Detailed Description	34
7.5.2 Constructor & Destructor Documentation	34
7.5.2.1 ~LinearBayesianFilter()	34
7.5.3 Member Function Documentation	34
7.5.3.1 CheckModel()	34
7.5.3.2 Correction()	34
7.5.3.3 Prediction()	34
7.5.3.4 Reset()	35

7.6 BayesianFiltering::NonlinearBayesianFilter Class Reference	35
7.6.1 Detailed Description	36
7.6.2 Constructor & Destructor Documentation	37
7.6.2.1 ~NonlinearBayesianFilter()	37
7.6.3 Member Function Documentation	37
7.6.3.1 Correction()	37
7.6.3.2 Prediction()	37
7.6.3.3 Reset()	37
7.7 BayesianFiltering::RobustSelfAdaptiveKalmanFilter Class Reference	38
7.7.1 Detailed Description	40
7.7.2 Constructor & Destructor Documentation	40
7.7.2.1 ~RobustSelfAdaptiveKalmanFilter()	40
7.7.3 Member Function Documentation	40
7.7.3.1 Correction()	40
7.7.3.2 Prediction()	41
7.7.3.3 Reset()	41
7.8 BayesianFiltering::SamplingImportanceResamplingFilter Class Reference	41
7.8.1 Detailed Description	44
7.8.2 Member Enumeration Documentation	44
7.8.2.1 EstimateType	44
7.8.3 Constructor & Destructor Documentation	45
7.8.3.1 ~SamplingImportanceResamplingFilter()	45
7.8.4 Member Function Documentation	45
7.8.4.1 ComputeESS()	45
7.8.4.2 Correction()	45
7.8.4.3 EnumToString()	45
7.8.4.4 NormalizeWeights()	46
7.8.4.5 Prediction()	46
7.8.4.6 Reset() [1/2]	46
7.8.4.7 Reset() [2/2]	46
7.8.5 Member Data Documentation	47
7.8.5.1 ESS	47
7.8.5.2 estimate_type	47
7.8.5.3 Nparticles	47
7.8.5.4 Wparticles	47
7.8.5.5 Xparticles	47
7.8.5.6 Yparticles	47
7.9 BayesianFiltering::StateSpaceLinearModel Class Reference	48
7.9.1 Detailed Description	49
7.9.2 Member Function Documentation	49
7.9.2.1 MeasurementModel()	49
7.9.2.2 ProcessModel()	49

7.9.2.3 SetDimensions()	50
7.9.3 Member Data Documentation	50
7.9.3.1 A	50
7.9.3.2 B	50
7.9.3.3 C	50
7.9.3.4 Nu	51
7.9.3.5 Nv	51
7.9.3.6 Nw	51
7.9.3.7 Nx	51
7.9.3.8 Ny	51
7.9.3.9 P0	51
7.9.3.10 Q	51
7.9.3.11 R	51
7.9.3.12 X0	52
7.10 BayesianFiltering::StateSpaceNonlinearModelBase Class Reference	52
7.10.1 Detailed Description	53
7.10.2 Constructor & Destructor Documentation	54
7.10.2.1 ~StateSpaceNonlinearModelBase()	54
7.10.3 Member Function Documentation	54
7.10.3.1 MeasurementModel()	54
7.10.3.2 ProcessModel()	54
7.10.3.3 SetDimensions()	54
7.10.3.4 UpdatedfdW()	55
7.10.3.5 UpdatedfdX()	55
7.10.3.6 UpdatedhdV()	56
7.10.3.7 UpdatedhdX()	56
7.10.4 Member Data Documentation	56
7.10.4.1 dfdW	56
7.10.4.2 dfdX	56
7.10.4.3 dhdV	57
7.10.4.4 dhdX	57
7.10.4.5 Nu	57
7.10.4.6 Nv	57
7.10.4.7 Nw	57
7.10.4.8 Nx	57
7.10.4.9 Ny	57
7.10.4.10 P0	57
7.10.4.11 Q	58
7.10.4.12 R	58
7.10.4.13 X0	58
7.11 BayesianFiltering::UnscentedKalmanFilter Class Reference	58
7.11.1 Detailed Description	60

7.11.2 Constructor & Destructor Documentation	60
7.11.2.1 <code>~UnscentedKalmanFilter()</code>	60
7.11.3 Member Function Documentation	61
7.11.3.1 <code>Correction()</code>	61
7.11.3.2 <code>Prediction()</code>	62
7.11.3.3 <code>Reset()</code>	62
8 File Documentation	63
8.1 <code>bayesianfiltering.hpp</code> File Reference	63
8.1.1 Macro Definition Documentation	65
8.1.1.1 <code>PRINT_EIGEN_MATRIX</code>	65
8.1.1.2 <code>PRINT_EIGEN_VECTOR</code>	65
8.1.1.3 <code>PRINT_LINE_DEBUG</code>	65
8.1.2 Function Documentation	66
8.1.2.1 <code>PrintEigenMatrixXd()</code>	66
8.1.2.2 <code>PrintEigenVectorXd()</code>	66
8.2 <code>bayesianfiltering.hpp</code>	66
8.3 <code>README.md</code> File Reference	78
Index	79

Chapter 1

About this library

This library is developed to be of simple use, requiring only knowledge on Bayesian filtering and object oriented C++ programming. It is contained in a single `bayesianfiltering.hpp` file.

It deals with both time varying linear and nonlinear discrete time models. For more details, see the documentation at `/doc` folder.

1.1 Dependencies

The library was developed with little dependence on other libraries or operating system. Some dependencies vary according to the demos provided. See the corresponding `Readme.md` file.

1.1.1 EIGEN

EIGEN is one of the most important libraries of linear algebra for C++. The Bayesian Filtering library relies strongly on it. Just install according to your system.

1.1.2 Doxygen (optional)

Doxygen is an optional installation, only if you want to update the documentation (html and/or pdf).

Installation can be done in different operational systems. See Doxygen project at <https://www.doxygen.nl/>

This library already has a Doxyfile configured at library root folder.

In the library root folder, run the following command:

```
$ doxygen
```

This will generate documentation in html pages (`doc/html`) and source files for latex/pdf version (`doc/latex`).

In `doc/html`, open `index.html` in any browser.

In `doc/latex`, in order to generate the pdf documentation just run:

```
$ make pdf
```

This will compile latex sources and generate `doc/latex/refman.pdf`

If latex and pdflatex are not installed in your system, install it.

For Linux, this can be accomplished by running:

```
$ sudo apt install texlive-latex-base  
$ sudo apt install texlive-fonts-recommended texlive-latex-extra
```

Or, if you have lots of disk space, just run:

```
$ sudo apt install texlive-full
```

1.2 Demos

At `demos` folder you can find the current usage examples, with folders related to the kind of platform. Explore it! They are well documented.

For instance, Sampling Importance Resampling (SIR) particle filter demo with 1000 particles generate the following figures:

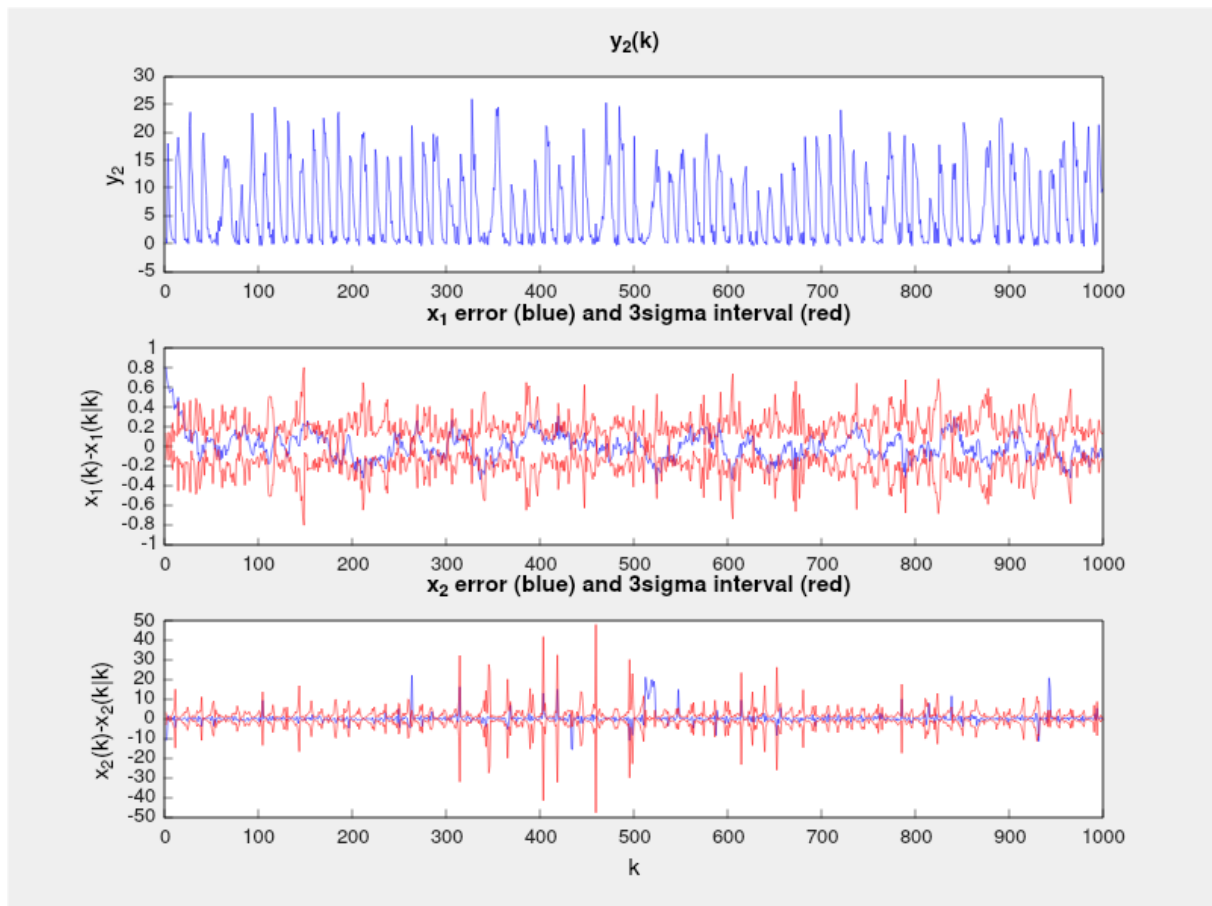


Figure 1.1 Figure 1

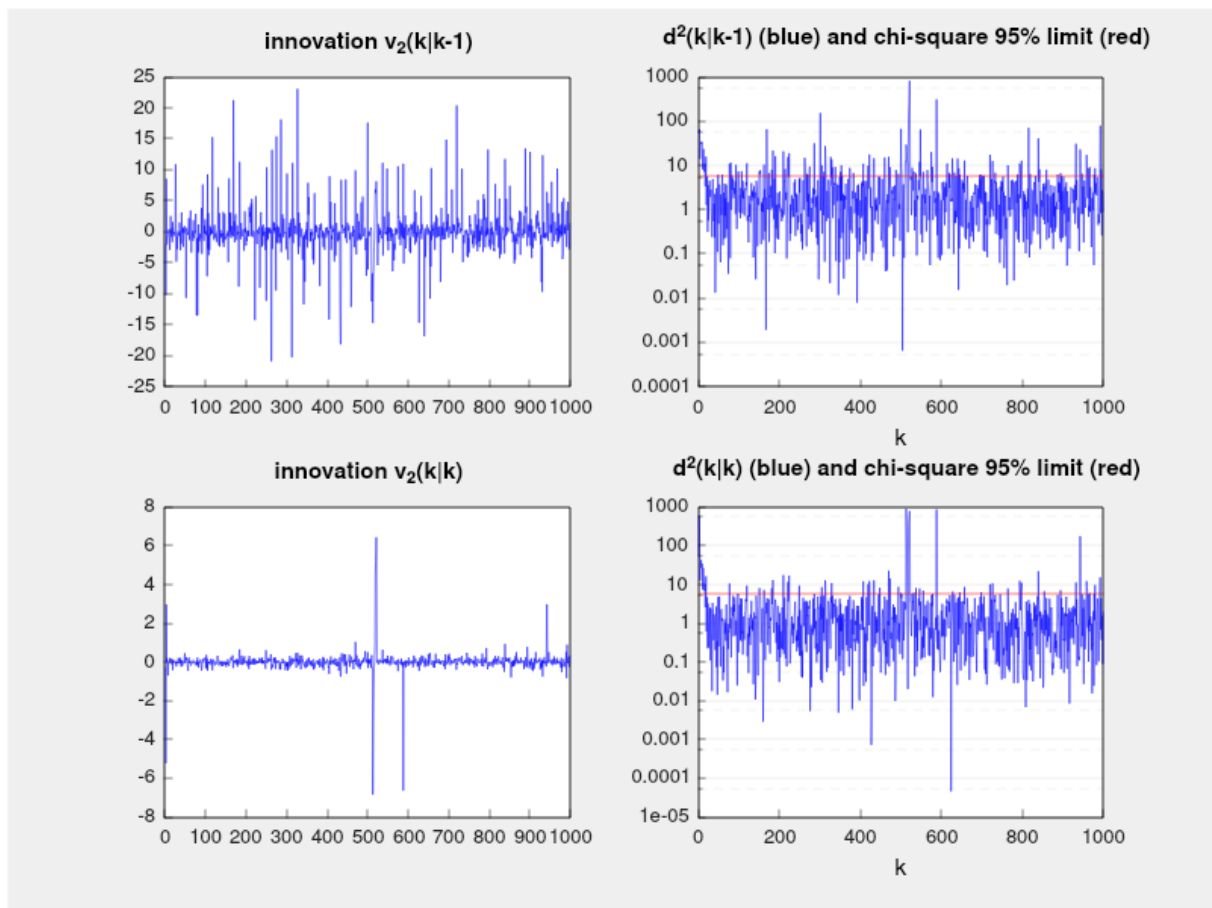


Figure 1.2 Figure 2

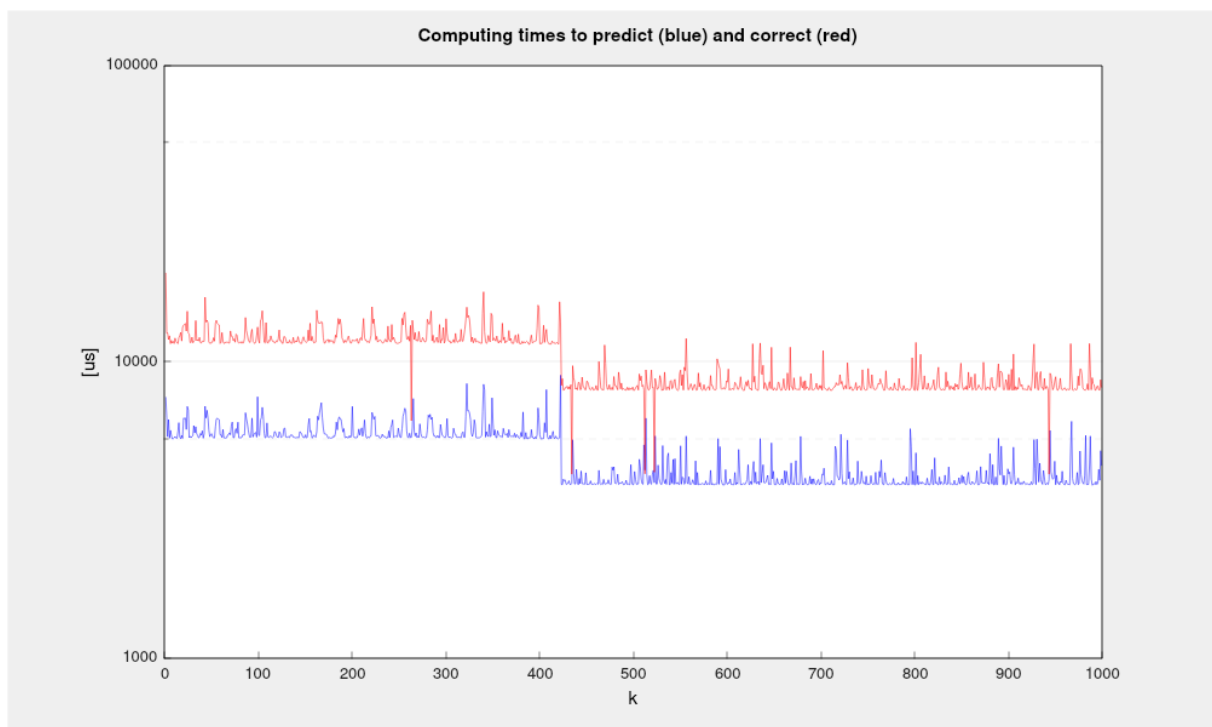


Figure 1.3 Figure 3

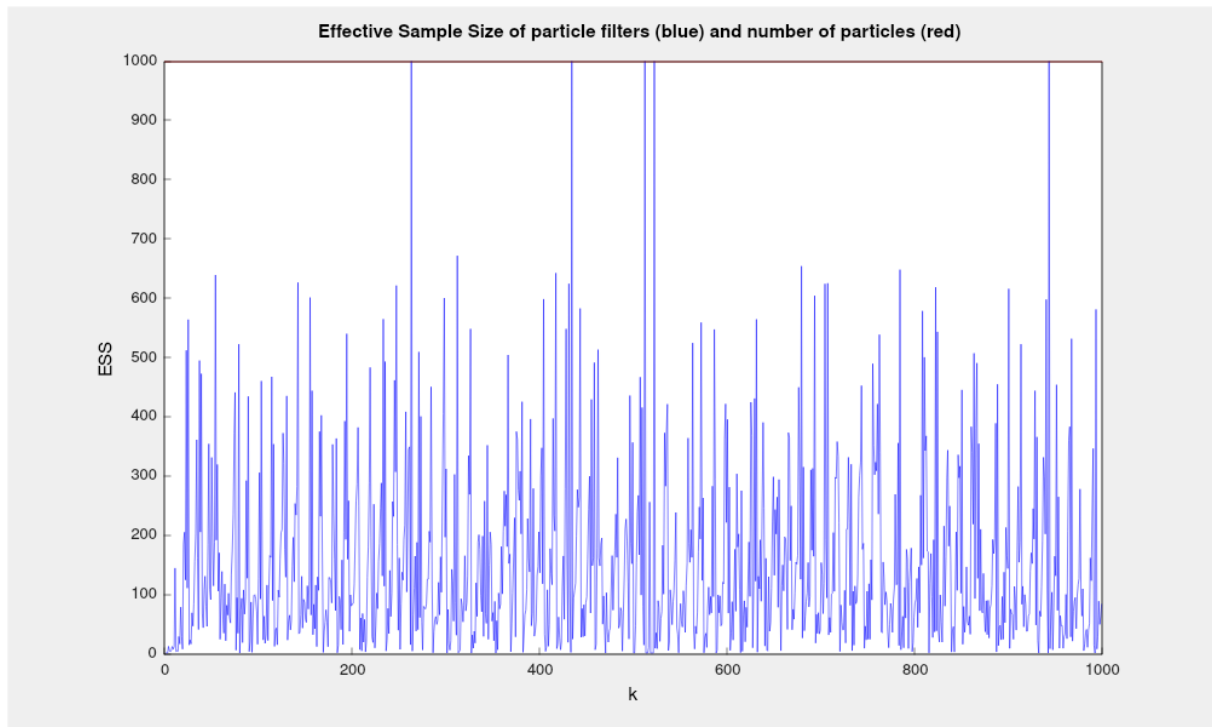


Figure 1.4 Figure 4

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

BayesianFiltering	13
---	----

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

BayesianFiltering::BayesianFilter	22
BayesianFiltering::LinearBayesianFilter	32
BayesianFiltering::AdaptiveKalmanFilter	17
BayesianFiltering::KalmanFilter	29
BayesianFiltering::RobustSelfAdaptiveKalmanFilter	38
BayesianFiltering::NonlinearBayesianFilter	35
BayesianFiltering::ExtendedKalmanFilter	25
BayesianFiltering::SamplingImportanceResamplingFilter	41
BayesianFiltering::UnscentedKalmanFilter	58
BayesianFiltering::StateSpaceLinearModel	48
BayesianFiltering::StateSpaceNonlinearModelBase	52

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BayesianFiltering::AdaptiveKalmanFilter	Adaptive Kalman filter, with $\mathbf{Q}$ and $\mathbf{R}$ matrices changing when every measurement comes according two user supplied learning parameters	17
BayesianFiltering::BayesianFilter	Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop	22
BayesianFiltering::ExtendedKalmanFilter	Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models	25
BayesianFiltering::KalmanFilter	Kalman filter, the best optimal unbiased estimator for stochastic linear state space models . . .	29
BayesianFiltering::LinearBayesianFilter	Base class of Bayesian filters for state space linear models. See derived classes	32
BayesianFiltering::NonlinearBayesianFilter	Base class of Bayesian filters for state space nonlinear models. See derived classes	35
BayesianFiltering::RobustSelfAdaptiveKalmanFilter	Robust Self Adaptive Kalman filter, with $\mathbf{Q}$ and $\mathbf{R}$ matrices changing when every measurement comes using covariance matching strategy	38
BayesianFiltering::SamplingImportanceResamplingFilter	Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models	41
BayesianFiltering::StateSpaceLinearModel	Represents the parameters of a discrete time, generic linear state space model: 48	
BayesianFiltering::StateSpaceNonlinearModelBase	Represents the parameters of a discrete time, generic nonlinear state space model: 52	
BayesianFiltering::UnscentedKalmanFilter	Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models	58

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

bayesianfiltering.hpp	63
---	----

Chapter 6

Namespace Documentation

6.1 BayesianFiltering Namespace Reference

Classes

- class [AdaptiveKalmanFilter](#)
Adaptive Kalman filter, with Q and R matrices changing when every measurement comes according two user supplied learning parameters.
- class [BayesianFilter](#)
Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop.
- class [ExtendedKalmanFilter](#)
Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models.
- class [KalmanFilter](#)
Kalman filter, the best optimal unbiased estimator for stochastic linear state space models.
- class [LinearBayesianFilter](#)
Base class of Bayesian filters for state space linear models. See derived classes.
- class [NonlinearBayesianFilter](#)
Base class of Bayesian filters for state space nonlinear models. See derived classes.
- class [RobustSelfAdaptiveKalmanFilter](#)
Robust Self Adaptive Kalman filter, with Q and R matrices changing when every measurement comes using covariance matching strategy.
- class [SamplingImportanceResamplingFilter](#)
Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models.
- class [StateSpaceLinearModel](#)
Represents the parameters of a discrete time, generic linear state space model:
- class [StateSpaceNonlinearModelBase](#)
Represents the parameters of a discrete time, generic nonlinear state space model:
- class [UnscentedKalmanFilter](#)
Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models.

Typedefs

- typedef std::pair< bool, std::string > [Result](#)
Encapsulated <bool,std::string> pair response of a function, with .first specifying status (success of failure) and .second the reason string in case of failure.

Functions

- [Result ResultFormat](#) (bool success, const std::string &message)
Format <bool,std::string> pair response with defined status and reason string.
- `template<typename ... Args>`
[Result ResultFormat](#) (bool success, const std::string &format, Args ... args)
Format <bool,std::string> pair response with defined status and reason string to be printf-like formatted.
- [Result MultivariateZeroMeanGaussianSampler](#) (Eigen::VectorXd &sample, Eigen::MatrixXd &covariance_↵
matrix)
Generate multivariate $sample \sim N(0, covariance_matrix)$.
- [Result MultivariateGaussianSampler](#) (Eigen::VectorXd &sample, Eigen::VectorXd &mean_vector, Eigen::↵
MatrixXd &covariance_matrix)
Generate multivariate $sample \sim N(mean_vector, covariance_matrix)$.
- double [UnivariateGaussianSampler](#) (double mean, double variance)
Generate univariate $sample \sim N(mean, variance)$.
- double [UnivariateUniformSampler](#) (double min_value, double max_value)
Generate univariate $sample \sim U(min_value, max_value)$.

6.1.1 Typedef Documentation

6.1.1.1 Result

```
typedef std::pair<bool, std::string> BayesianFiltering::Result
```

Encapsulated <bool,std::string> pair response of a function, with .first specifying status (success of failure) and .second the reason string in case of failure.

6.1.2 Function Documentation

6.1.2.1 MultivariateGaussianSampler()

```
Result BayesianFiltering::MultivariateGaussianSampler (
    Eigen::VectorXd & sample,
    Eigen::VectorXd & mean_vector,
    Eigen::MatrixXd & covariance_matrix )
```

Generate multivariate $sample \sim N(mean_vector, covariance_matrix)$.

Parameters

out	<i>sample</i>	will be filled with the outcome
in	<i>mean_vector</i>	as above
in	<i>covariance_matrix</i>	as above

Returns

Result type

6.1.2.2 MultivariateZeroMeanGaussianSampler()

```
Result BayesianFiltering::MultivariateZeroMeanGaussianSampler (
    Eigen::VectorXd & sample,
    Eigen::MatrixXd & covariance_matrix )
```

Generate multivariate sample $\sim N(0, \text{covariance_matrix})$.

Parameters

out	<i>sample</i>	will be filled with the outcome
in	<i>covariance_matrix</i>	as above

Returns

Result type

6.1.2.3 ResultFormat() [1/2]

```
template<typename ... Args>
Result BayesianFiltering::ResultFormat (
    bool success,
    const std::string & format,
    Args ... args )
```

Format <bool,std::string> pair response with defined status and reason string to be printf-like formatted.

Parameters

in	<i>success</i>	bool indicating status
in	<i>format</i>	formatting string
in	<i>args</i>	sequence of variable arguments for message formatting

6.1.2.4 ResultFormat() [2/2]

```
Result BayesianFiltering::ResultFormat (
    bool success,
    const std::string & message ) [inline]
```

Format <bool,std::string> pair response with defined status and reason string.

Parameters

in	<i>success</i>	bool indicating status
in	<i>message</i>	string message

6.1.2.5 UnivariateGaussianSampler()

```
double BayesianFiltering::UnivariateGaussianSampler (
    double mean,
    double variance )
```

Generate univariate $\text{sample} \sim \mathcal{N}(\text{mean}, \text{variance})$.

Parameters

in	<i>mean</i>	as above
in	<i>variance</i>	as above

Returns

sample

6.1.2.6 UnivariateUniformSampler()

```
double BayesianFiltering::UnivariateUniformSampler (
    double min_value,
    double max_value )
```

Generate univariate $\text{sample} \sim \mathcal{U}(\text{min_value}, \text{max_value})$.

Parameters

in	<i>min_value</i>	as above
in	<i>max_value</i>	as above

Returns

sample

Chapter 7

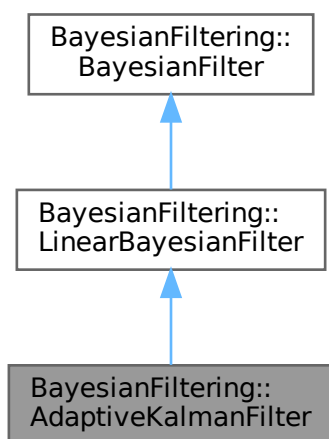
Class Documentation

7.1 BayesianFiltering::AdaptiveKalmanFilter Class Reference

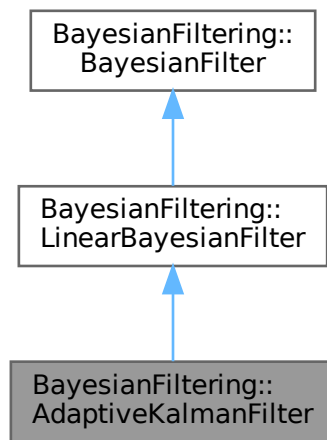
Adaptive Kalman filter, with Q and R matrices changing when every measurement comes according two user supplied learning parameters.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::AdaptiveKalmanFilter:



Collaboration diagram for BayesianFiltering::AdaptiveKalmanFilter:



Public Member Functions

- virtual [~AdaptiveKalmanFilter](#) ()
- void [Reset](#) ([StateSpaceLinearModel](#) &linear_model, double _alpha, double _beta)
Reset filter with user supplied filter specific parameters. Must be called before the prediction/correction cycle.
- void [Reset](#) ([StateSpaceLinearModel](#) &linear_model) override
Reset filter with default filter specific parameters ($\alpha = 0.95$ and $\beta = 0.99$). Must be called before the prediction/correction cycle.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, [StateSpaceLinearModel](#) &linear_model) override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model $linear_model$. In case of time varying systems, $linear_model$ class members should be updated before calling this procedure.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, [StateSpaceLinearModel](#) &linear_model) override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model $linear_model$. In case of time varying systems, $linear_model$ class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- virtual [~LinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Public Attributes

- double [alpha](#)
User supplied learning parameter for updating R measurement noise covariance matrix. Must be in interval $]0,1]$, with 1 leading to fixed R matrix.
- double [beta](#)
User supplied learning parameter for updating Q process noise covariance matrix. Must be in interval $]0,1]$, with 1 leading to fixed Q matrix.

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- Eigen::VectorXd [X](#)
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $\hat{y}(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - \hat{y}(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $\hat{y}(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - \hat{y}(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

Additional Inherited Members**Protected Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)**

- [Result CheckModel](#) ([StateSpaceLinearModel](#) &linear_model)

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.1.1 Detailed Description

Adaptive Kalman filter, with Q and R matrices changing when every measurement comes according two user supplied learning parameters.

This implementation is specific for discrete time, continuous state models.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 ~AdaptiveKalmanFilter()

```
virtual BayesianFiltering::AdaptiveKalmanFilter::~AdaptiveKalmanFilter ( ) [inline], [virtual]
```

7.1.3 Member Function Documentation

7.1.3.1 Correction()

```
Result BayesianFiltering::AdaptiveKalmanFilter::Correction (
    Eigen::VectorXd & Y,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	<i>Y</i>	measurement vector of the linear system process model
in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.1.3.2 Prediction()

```
Result BayesianFiltering::AdaptiveKalmanFilter::Prediction (
    Eigen::VectorXd & U,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	<i>U</i>	input vector of the linear system process model
in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.1.3.3 Reset() [1/2]

```
void BayesianFiltering::AdaptiveKalmanFilter::Reset (
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Reset filter with default filter specific parameters (`alpha = 0.95` and `beta = 0.99`). Must be called before the prediction/correction cycle.

Parameters

in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run
----	---------------------	--

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.1.3.4 Reset() [2/2]

```
void BayesianFiltering::AdaptiveKalmanFilter::Reset (
    StateSpaceLinearModel & linear_model,
    double _alpha,
    double _beta ) [inline]
```

Reset filter with user supplied filter specific parameters. Must be called before the prediction/correction cycle.

Parameters

in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run
in	<i>_alpha</i>	user provided value for alpha member variable
in	<i>_beta</i>	user provided value for beta member variable

7.1.4 Member Data Documentation

7.1.4.1 alpha

```
double BayesianFiltering::AdaptiveKalmanFilter::alpha
```

User supplied learning parameter for updating R measurement noise covariance matrix. Must be in interval $]0,1]$, with 1 leading to fixed R matrix.

7.1.4.2 beta

```
double BayesianFiltering::AdaptiveKalmanFilter::beta
```

User supplied learning parameter for updating Q process noise covariance matrix. Must be in interval $]0,1]$, with 1 leading to fixed Q matrix.

The documentation for this class was generated from the following file:

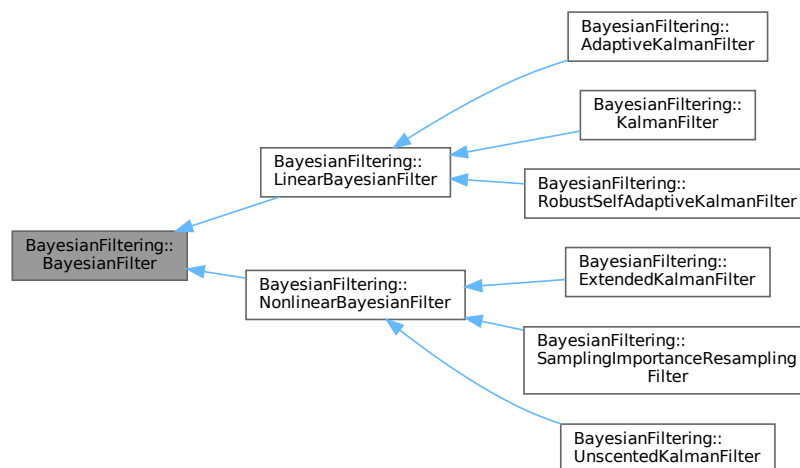
- [bayesianfiltering.hpp](#)

7.2 BayesianFiltering::BayesianFilter Class Reference

Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::BayesianFilter:



Public Member Functions

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Public Attributes

- Eigen::VectorXd [X](#)
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $\hat{y}(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - \hat{y}(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $\hat{y}(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - \hat{y}(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

Protected Member Functions

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.2.1 Detailed Description

Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop.

1. Reset
2. Prediction: compute $x(k|k-1)$ using $x(k-1)$ and input $u(k)$
3. Correction, in case of measurement available: compute $x(k|k)$ using $x(k|k-1)$ and measurement vector $y(k)$;
4. In case of filter divergence, perform necessary actions and go back to step 1. Otherwise, go back to step 2.

As it can be seen, as described above correction is optional, which allows filter continuing by prediction. Internal member variables present the current filter state.

System model object should be not used in other purposes other than to be used as input to the filter. For the case of linear time varying models, it can have their parameters directly updated just before calling prediction/correction methods, as a way to deal with time varying systems. For the case of nonlinear time varying models, parameters updating is done in derived model class, where the methods are called from inside filter object. Thus, model updating is done in derived class, being not necessary to be done by the user.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 BayesianFilter()

```
BayesianFiltering::BayesianFilter::BayesianFilter ( ) [inline]
```

7.2.2.2 ~BayesianFilter()

```
virtual BayesianFiltering::BayesianFilter::~~BayesianFilter ( ) [inline], [virtual]
```

7.2.3 Member Function Documentation

7.2.3.1 Reset()

```
void BayesianFiltering::BayesianFilter::Reset (
    Eigen::VectorXd & X0,
    Eigen::MatrixXd & P0,
    int Nx,
    int Ny ) [inline], [protected]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	$X0$	initial state vector mean value of $x(0)$ distribution
in	$P0$	initial state error covariance matrix of $x(0)$ distribution
in	Nx	Number of state variables in vector X
in	Ny	Number of measurements in vector Y

7.2.4 Member Data Documentation**7.2.4.1 d2_corrected**

```
double BayesianFiltering::BayesianFilter::d2_corrected
```

Corrected innovation Mahalanobis distance, allowing to verify a filter os optimistic ou pessimistic, and divergence.

7.2.4.2 d2_predicted

```
double BayesianFiltering::BayesianFilter::d2_predicted
```

Predicted innovation Mahalanobis distance, allowing to verify a filter os optimistic ou pessimistic, and divergence.

7.2.4.3 P

```
Eigen::MatrixXd BayesianFiltering::BayesianFilter::P
```

State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.

7.2.4.4 S_corrected

```
Eigen::MatrixXd BayesianFiltering::BayesianFilter::S_corrected
```

Corrected innovation covariance matrix.

7.2.4.5 S_predicted

```
Eigen::MatrixXd BayesianFiltering::BayesianFilter::S_predicted
```

Predicted innovation covariance matrix.

7.2.4.6 V_corrected

```
Eigen::VectorXd BayesianFiltering::BayesianFilter::V_corrected
```

Corrected innovation $y(k) - y(k|k)$.

7.2.4.7 V_predicted

`Eigen::VectorXd BayesianFiltering::BayesianFilter::V_predicted`

Predicted innovation $\hat{y}(k) - \hat{y}(k|k-1)$.

7.2.4.8 X

`Eigen::VectorXd BayesianFiltering::BayesianFilter::X`

State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.

7.2.4.9 Y_corrected

`Eigen::VectorXd BayesianFiltering::BayesianFilter::Y_corrected`

Corrected measurement $y(k|k)$.

7.2.4.10 Y_predicted

`Eigen::VectorXd BayesianFiltering::BayesianFilter::Y_predicted`

Predicted measurement $\hat{y}(k|k-1)$.

The documentation for this class was generated from the following file:

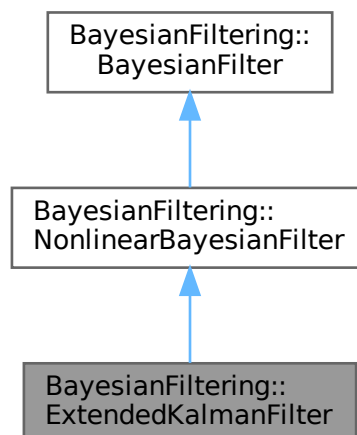
- [bayesianfiltering.hpp](#)

7.3 BayesianFiltering::ExtendedKalmanFilter Class Reference

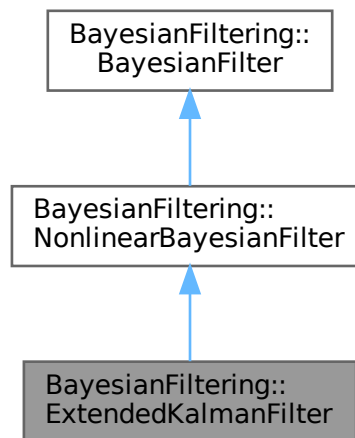
Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::ExtendedKalmanFilter:



Collaboration diagram for BayesianFiltering::ExtendedKalmanFilter:



Public Member Functions

- virtual [~ExtendedKalmanFilter](#) ()
- void [Reset](#) ([StateSpaceNonlinearModelBase](#) &nonlinear_model) override
Reset filter, first procedure before prediction/correction.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `nonlinear_model`. This procedure calls methods of the derived class object `nonlinear_model` in order to update model parameters. So there is no need the user to do that.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::NonlinearBayesianFilter](#)

- virtual [~NonlinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Additional Inherited Members

Public Attributes inherited from BayesianFiltering::BayesianFilter

- Eigen::VectorXd [X](#)
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $\hat{y}(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - \hat{y}(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $\hat{y}(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - \hat{y}(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.

Protected Member Functions inherited from BayesianFiltering::BayesianFilter

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.3.1 Detailed Description

Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models. This implementation is specific for discrete time, continuous state models.

7.3.2 Constructor & Destructor Documentation

7.3.2.1 ~ExtendedKalmanFilter()

```
virtual BayesianFiltering::ExtendedKalmanFilter::~ExtendedKalmanFilter ( ) [inline], [virtual]
```

7.3.3 Member Function Documentation

7.3.3.1 Correction()

```
Result BayesianFiltering::ExtendedKalmanFilter::Correction (
    Eigen::VectorXd & Y,
    double t,
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	t	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.3.3.2 Prediction()

```
Result BayesianFiltering::ExtendedKalmanFilter::Prediction (
    Eigen::VectorXd & U,
    double t,
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `nonlinear_model`. This procedure calls methods of the derived class object `nonlinear_model` in order to update model parameters. So there is no need the user to do that.

Parameters

in	U	input vector of the linear system process model
in	t	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.3.3.3 Reset()

```
void BayesianFiltering::ExtendedKalmanFilter::Reset (
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run
----	------------------------	---

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

The documentation for this class was generated from the following file:

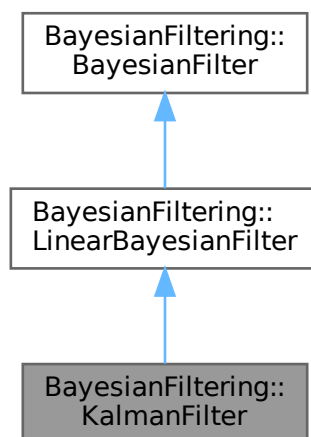
- [bayesianfiltering.hpp](#)

7.4 BayesianFiltering::KalmanFilter Class Reference

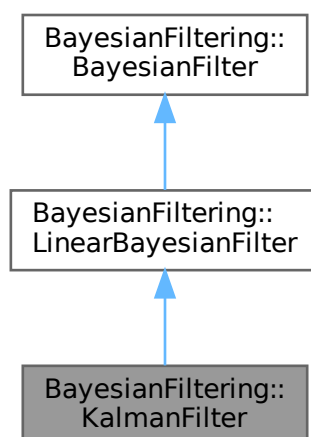
Kalman filter, the best optimal unbiased estimator for stochastic linear state space models.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::KalmanFilter:



Collaboration diagram for BayesianFiltering::KalmanFilter:



Public Member Functions

- virtual [~KalmanFilter](#) ()
- void [Reset](#) ([StateSpaceLinearModel](#) &linear_model) override
Reset filter, first procedure before prediction/correction.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, [StateSpaceLinearModel](#) &linear_model) override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, [StateSpaceLinearModel](#) &linear_model) override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- virtual [~LinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Additional Inherited Members

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- [Eigen::VectorXd](#) [X](#)
State vector predictor $x(k|k-1)$ or estimate $x(k|k)$, depending on which stage the filter is.
- [Eigen::MatrixXd](#) [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- [Eigen::VectorXd](#) [Y_predicted](#)
Predicted measurement $y(k|k-1)$.
- [Eigen::VectorXd](#) [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- [Eigen::MatrixXd](#) [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- [Eigen::VectorXd](#) [Y_corrected](#)
Corrected measurement $y(k|k)$.
- [Eigen::VectorXd](#) [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- [Eigen::MatrixXd](#) [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

Protected Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- [Result CheckModel](#) ([StateSpaceLinearModel](#) &linear_model)

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.4.1 Detailed Description

Kalman filter, the best optimal unbiased estimator for stochastic linear state space models.

This implementation is specific for discrete time, continuous state models.

7.4.2 Constructor & Destructor Documentation

7.4.2.1 ~KalmanFilter()

```
virtual BayesianFiltering::KalmanFilter::~~KalmanFilter ( ) [inline], [virtual]
```

7.4.3 Member Function Documentation

7.4.3.1 Correction()

```
Result BayesianFiltering::KalmanFilter::Correction (
    Eigen::VectorXd & Y,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.4.3.2 Prediction()

```
Result BayesianFiltering::KalmanFilter::Prediction (
    Eigen::VectorXd & U,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables $\hat{X}$ and $\hat{P}$ based on input vector U and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	U	input vector of the linear system process model
in	<code>linear_model</code>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.4.3.3 Reset()

```
void BayesianFiltering::KalmanFilter::Reset (
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	<code>linear_model</code>	StateSpaceLinearModel object for which the filter will run
----	---------------------------	--

Implements [BayesianFiltering::LinearBayesianFilter](#).

The documentation for this class was generated from the following file:

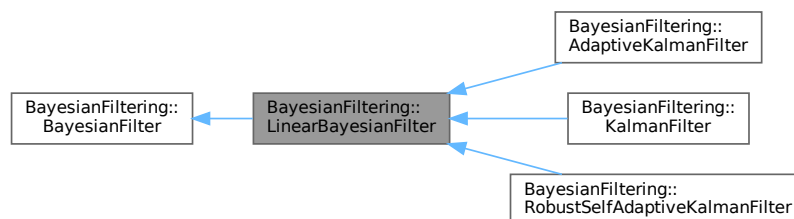
- [bayesianfiltering.hpp](#)

7.5 BayesianFiltering::LinearBayesianFilter Class Reference

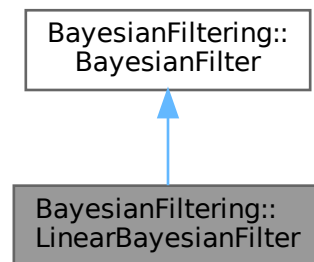
Base class of Bayesian filters for state space linear models. See derived classes.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::LinearBayesianFilter:



Collaboration diagram for BayesianFiltering::LinearBayesianFilter:



Public Member Functions

- virtual `~LinearBayesianFilter` ()
- virtual `Result Prediction` (Eigen::VectorXd &U, `StateSpaceLinearModel` &linear_model)=0
- virtual `Result Correction` (Eigen::VectorXd &Y, `StateSpaceLinearModel` &linear_model)=0
- virtual void `Reset` (`StateSpaceLinearModel` &linear_model)=0

Public Member Functions inherited from `BayesianFiltering::BayesianFilter`

- `BayesianFilter` ()
- virtual `~BayesianFilter` ()

Protected Member Functions

- `Result CheckModel` (`StateSpaceLinearModel` &linear_model)

Protected Member Functions inherited from `BayesianFiltering::BayesianFilter`

- void `Reset` (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

Additional Inherited Members

Public Attributes inherited from `BayesianFiltering::BayesianFilter`

- Eigen::VectorXd `X`
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd `P`
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd `Y_predicted`
Predicted measurement $\hat{y}(k|k-1)$.

- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $y(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

7.5.1 Detailed Description

Base class of Bayesian filters for state space linear models. See derived classes.

7.5.2 Constructor & Destructor Documentation

7.5.2.1 ~LinearBayesianFilter()

```
virtual BayesianFiltering::LinearBayesianFilter::~~LinearBayesianFilter ( ) [inline], [virtual]
```

7.5.3 Member Function Documentation

7.5.3.1 CheckModel()

```
Result BayesianFiltering::LinearBayesianFilter::CheckModel (
    StateSpaceLinearModel & linear_model ) [inline], [protected]
```

7.5.3.2 Correction()

```
virtual Result BayesianFiltering::LinearBayesianFilter::Correction (
    Eigen::VectorXd & Y,
    StateSpaceLinearModel & linear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::KalmanFilter](#), [BayesianFiltering::AdaptiveKalmanFilter](#), and [BayesianFiltering::RobustSelfAdaptiveKalmanFilter](#).

7.5.3.3 Prediction()

```
virtual Result BayesianFiltering::LinearBayesianFilter::Prediction (
    Eigen::VectorXd & U,
    StateSpaceLinearModel & linear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::KalmanFilter](#), [BayesianFiltering::AdaptiveKalmanFilter](#), and [BayesianFiltering::RobustSelfAdaptiveKalmanFilter](#).

7.5.3.4 Reset()

```
virtual void BayesianFiltering::LinearBayesianFilter::Reset (
    StateSpaceLinearModel & linear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::KalmanFilter](#), [BayesianFiltering::AdaptiveKalmanFilter](#), and [BayesianFiltering::RobustSelfAdaptiveFilter](#).

The documentation for this class was generated from the following file:

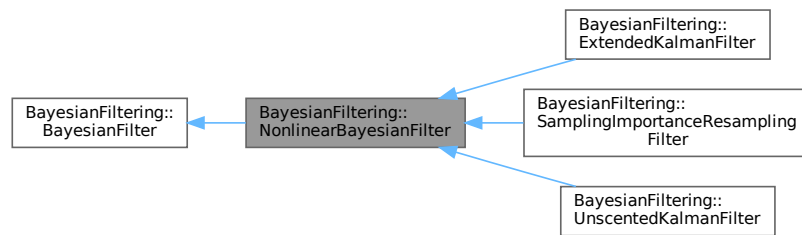
- [bayesianfiltering.hpp](#)

7.6 BayesianFiltering::NonlinearBayesianFilter Class Reference

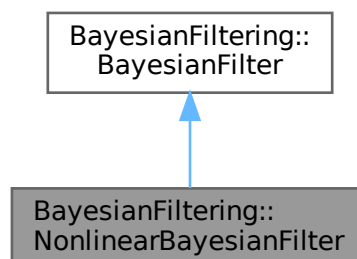
Base class of Bayesian filters for state space nonlinear models. See derived classes.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::NonlinearBayesianFilter:



Collaboration diagram for BayesianFiltering::NonlinearBayesianFilter:



Public Member Functions

- virtual [~NonlinearBayesianFilter](#) ()
- virtual [Result Prediction](#) (Eigen::VectorXd &U, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model)=0
- virtual [Result Correction](#) (Eigen::VectorXd &Y, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model)=0
- virtual void [Reset](#) ([StateSpaceNonlinearModelBase](#) &nonlinear_model)=0

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Additional Inherited Members

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- Eigen::VectorXd [X](#)
State vector predictor $x(k|k-1)$ or estimate $x(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $y(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $y(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.6.1 Detailed Description

Base class of Bayesian filters for state space nonlinear models. See derived classes.

7.6.2 Constructor & Destructor Documentation

7.6.2.1 ~NonlinearBayesianFilter()

```
virtual BayesianFiltering::NonlinearBayesianFilter::~~NonlinearBayesianFilter ( ) [inline],  
[virtual]
```

7.6.3 Member Function Documentation

7.6.3.1 Correction()

```
virtual Result BayesianFiltering::NonlinearBayesianFilter::Correction (   
    Eigen::VectorXd & Y,  
    double t,  
    StateSpaceNonlinearModelBase & nonlinear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::ExtendedKalmanFilter](#), [BayesianFiltering::UnscentedKalmanFilter](#), and [BayesianFiltering::SamplingImportanceResamplingFilter](#).

7.6.3.2 Prediction()

```
virtual Result BayesianFiltering::NonlinearBayesianFilter::Prediction (   
    Eigen::VectorXd & U,  
    double t,  
    StateSpaceNonlinearModelBase & nonlinear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::ExtendedKalmanFilter](#), [BayesianFiltering::UnscentedKalmanFilter](#), and [BayesianFiltering::SamplingImportanceResamplingFilter](#).

7.6.3.3 Reset()

```
virtual void BayesianFiltering::NonlinearBayesianFilter::Reset (   
    StateSpaceNonlinearModelBase & nonlinear_model ) [pure virtual]
```

Implemented in [BayesianFiltering::ExtendedKalmanFilter](#), [BayesianFiltering::UnscentedKalmanFilter](#), and [BayesianFiltering::SamplingImportanceResamplingFilter](#).

The documentation for this class was generated from the following file:

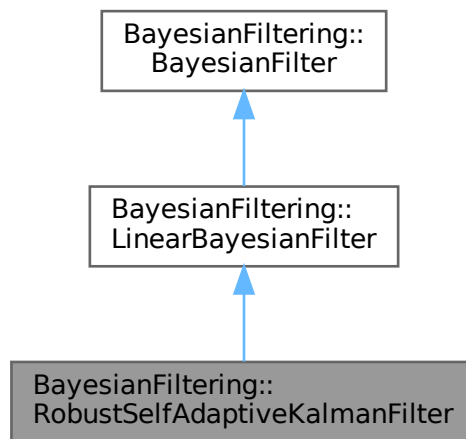
- [bayesianfiltering.hpp](#)

7.7 BayesianFiltering::RobustSelfAdaptiveKalmanFilter Class Reference

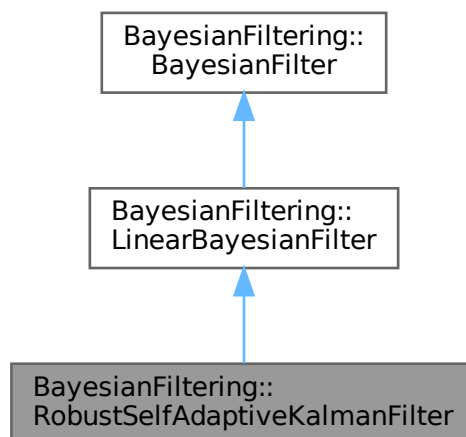
Robust Self Adaptive Kalman filter, with Q and R matrices changing when every measurement comes using covariance matching strategy.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::RobustSelfAdaptiveKalmanFilter:



Collaboration diagram for BayesianFiltering::RobustSelfAdaptiveKalmanFilter:



Public Member Functions

- virtual [~RobustSelfAdaptiveKalmanFilter](#) ()
- void [Reset](#) ([StateSpaceLinearModel](#) &linear_model) override
Reset filter, first procedure before prediction/correction.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, [StateSpaceLinearModel](#) &linear_model) override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, [StateSpaceLinearModel](#) &linear_model) override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- virtual [~LinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Additional Inherited Members

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- [Eigen::VectorXd](#) [X](#)
State vector predictor $x(k|k-1)$ or estimate $x(k|k)$, depending on which stage the filter is.
- [Eigen::MatrixXd](#) [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- [Eigen::VectorXd](#) [Y_predicted](#)
Predicted measurement $y(k|k-1)$.
- [Eigen::VectorXd](#) [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- [Eigen::MatrixXd](#) [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- [Eigen::VectorXd](#) [Y_corrected](#)
Corrected measurement $y(k|k)$.
- [Eigen::VectorXd](#) [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- [Eigen::MatrixXd](#) [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

Protected Member Functions inherited from [BayesianFiltering::LinearBayesianFilter](#)

- [Result CheckModel](#) ([StateSpaceLinearModel](#) &linear_model)

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.7.1 Detailed Description

Robust Self Adaptive Kalman filter, with Q and R matrices changing when every measurement comes using covariance matching strategy.

This implementation is specific for discrete time, continuous state models.

From: Chen Y-W, Tu K-M. Robust self-adaptive Kalman filter with application in target tracking. Measurement and Control. 2022;55(9-10):935-944. doi:10.1177/00202940221083548

7.7.2 Constructor & Destructor Documentation

7.7.2.1 `~RobustSelfAdaptiveKalmanFilter()`

```
virtual BayesianFiltering::RobustSelfAdaptiveKalmanFilter::~~RobustSelfAdaptiveKalmanFilter ( )
[inline], [virtual]
```

7.7.3 Member Function Documentation

7.7.3.1 `Correction()`

```
Result BayesianFiltering::RobustSelfAdaptiveKalmanFilter::Correction (
    Eigen::VectorXd & Y,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables $\hat{X}$ and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	<code>linear_model</code>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.7.3.2 Prediction()

```
Result BayesianFiltering::RobustSelfAdaptiveKalmanFilter::Prediction (
    Eigen::VectorXd & U,
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables $\mathbf{X}$ and $\mathbf{P}$ based on input vector $\mathbf{U}$ and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Parameters

in	<i>U</i>	input vector of the linear system process model
in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run

Implements [BayesianFiltering::LinearBayesianFilter](#).

7.7.3.3 Reset()

```
void BayesianFiltering::RobustSelfAdaptiveKalmanFilter::Reset (
    StateSpaceLinearModel & linear_model ) [inline], [override], [virtual]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	<i>linear_model</i>	StateSpaceLinearModel object for which the filter will run
----	---------------------	--

Implements [BayesianFiltering::LinearBayesianFilter](#).

The documentation for this class was generated from the following file:

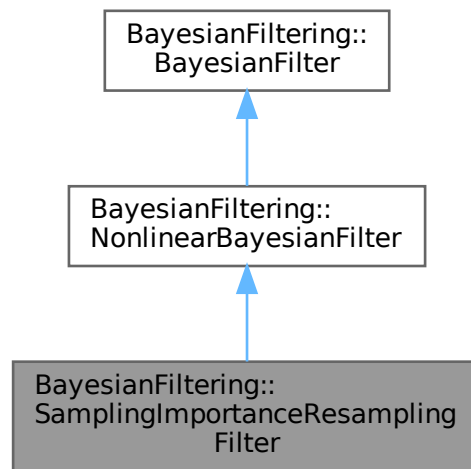
- [bayesianfiltering.hpp](#)

7.8 BayesianFiltering::SamplingImportanceResamplingFilter Class Reference

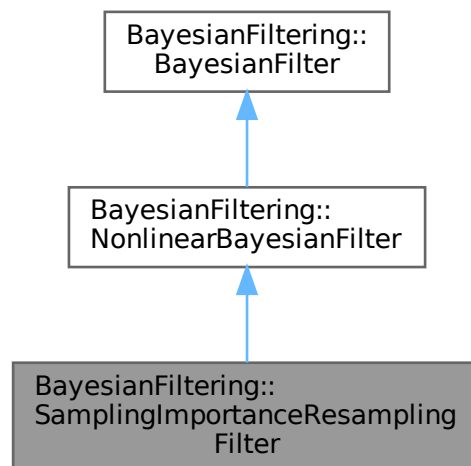
Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::SamplingImportanceResamplingFilter:



Collaboration diagram for BayesianFiltering::SamplingImportanceResamplingFilter:



Public Types

- enum class [EstimateType](#) { [MaximumAPosteriori](#) , [ConditionalMean](#) }
Enum of type of estimate computed by the filter at x vector.

Public Member Functions

- virtual [~SamplingImportanceResamplingFilter](#) ()
- void [Reset](#) ([StateSpaceNonlinearModelBase](#) &nonlinear_model, long int _Nparticles, [EstimateType](#) _estimate_type)

Reset filter with user supplied filter specific parameters. Must be called before the prediction/correction cycle.
- void [Reset](#) ([StateSpaceNonlinearModelBase](#) &nonlinear_model) override

Reset filter with default filter specific parameters (Nparticles = 1000 and estimate_type = [EstimateType::Conditional](#)). Must be called before the prediction/correction cycle.
- [Result NormalizeWeights](#) ()

Normalize particle's weight such that $\sum(w_{particles}) = 1.0$.
- [Result ComputeESS](#) ()

Update ESS (Effective Sample Size) member variable.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override

Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `nonlinear_model`. This procedure calls methods of the derived class object `nonlinear_model` in order to update model parameters. So there is no need the user to do that.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::NonlinearBayesianFilter](#)

- virtual [~NonlinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Static Public Member Functions

- static const char * [EnumToString](#) ([EstimateType](#) x)

Generate text description Enum of type of estimate computed by the filter at X vector.

Public Attributes

- [EstimateType](#) estimate_type

Estimation type to be provided by the filter at X vector.
- long int [Nparticles](#)

Number of particles.
- [Eigen::MatrixXd](#) [Xparticles](#)

X particles, one per col
- [Eigen::MatrixXd](#) [Yparticles](#)

Y particles, one per col
- [Eigen::VectorXd](#) [Wparticles](#)

Weight for each particle, set to $1/Nparticles$ after resampling.
- double [ESS](#)

Effective Sample Size, health of particles distribution. Read after filter correction.

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- Eigen::VectorXd [X](#)
State vector predictor $x(k|k-1)$ or estimate $x(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $y(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - y(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $y(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - y(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic ou pessimistic, and divergence.

Additional Inherited Members

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.8.1 Detailed Description

Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models.

This implementation is specific for discrete time, continuous state models.

Follows algorithm described at: M. S. Arulampalam, S. Maskell, N. Gordon and T. Clapp, "A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking", in IEEE Transactions on Signal Processing, vol. 50, no. 2, pp. 174-188, Feb. 2002, doi: 10.1109/78.978374.

7.8.2 Member Enumeration Documentation

7.8.2.1 EstimateType

```
enum class BayesianFiltering::SamplingImportanceResamplingFilter::EstimateType [strong]
```

Enum of type of estimate computed by the filter at X vector.

Enumerator

MaximumAPosteriori	Maximum a Posteriori (MAP) estimate.
ConditionalMean	Conditional mean estimate.

7.8.3 Constructor & Destructor Documentation

7.8.3.1 ~SamplingImportanceResamplingFilter()

```
virtual BayesianFiltering::SamplingImportanceResamplingFilter::~SamplingImportanceResamplingFilter ( ) [inline], [virtual]
```

7.8.4 Member Function Documentation

7.8.4.1 ComputeESS()

```
Result BayesianFiltering::SamplingImportanceResamplingFilter::ComputeESS ( ) [inline]
```

Update ESS (Effective Sample Size) member variable.

7.8.4.2 Correction()

```
Result BayesianFiltering::SamplingImportanceResamplingFilter::Correction (
    Eigen::VectorXd & Y,
    double t,
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Correction procedure, which will update (in place) the internal member variables **X** and **P** based on measurement vector **Y** and model **linear_model**. In case of time varying systems, **linear_model** class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	t	time variable, for time varying systems
in	nonlinear_model	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.8.4.3 EnumToString()

```
static const char * BayesianFiltering::SamplingImportanceResamplingFilter::EnumToString (
    EstimateType x ) [inline], [static]
```

Generate text description Enum of type of estimate computed by the filter at **X** vector.

7.8.4.4 NormalizeWeights()

Result BayesianFiltering::SamplingImportanceResamplingFilter::NormalizeWeights () [inline]

Normalize particle's weight such that $\sum(w_{particles}) = 1.0$.

7.8.4.5 Prediction()

Result BayesianFiltering::SamplingImportanceResamplingFilter::Prediction (Eigen::VectorXd & *U*, double *t*, StateSpaceNonlinearModelBase & *nonlinear_model*) [inline], [override], [virtual]

Prediction procedure, which will update (in place) the internal member variables *X* and *P* based on input vector *U* and model *nonlinear_model*. This procedure calls methods of the derived class object *nonlinear_model* in order to update model parameters. So there is no need the user to do that.

Parameters

in	<i>U</i>	input vector of the linear system process model
in	<i>t</i>	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements BayesianFiltering::NonlinearBayesianFilter.

7.8.4.6 Reset() [1/2]

void BayesianFiltering::SamplingImportanceResamplingFilter::Reset (StateSpaceNonlinearModelBase & *nonlinear_model*) [inline], [override], [virtual]

Reset filter with default filter specific parameters (*Nparticles* = 1000 and *estimate_type* = EstimateType::ConditionalMean). Must be called before the prediction/correction cycle.

Parameters

in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase object for which the filter will run
----	------------------------	---

Implements BayesianFiltering::NonlinearBayesianFilter.

7.8.4.7 Reset() [2/2]

void BayesianFiltering::SamplingImportanceResamplingFilter::Reset (StateSpaceNonlinearModelBase & *nonlinear_model*, long int *_Nparticles*, EstimateType *_estimate_type*) [inline]

Reset filter with user supplied filter specific parameters. Must be called before the prediction/correction cycle.

Parameters

in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase object for which the filter will run
in	<i>_Nparticles</i>	user provided value for Nparticles member variable
in	<i>_estimate_type</i>	user provided value for estimate_type member variable

7.8.5 Member Data Documentation

7.8.5.1 ESS

`double BayesianFiltering::SamplingImportanceResamplingFilter::ESS`

Effective Sample Size, health of particles distribution. Read after filter correction.

7.8.5.2 estimate_type

[EstimateType](#) `BayesianFiltering::SamplingImportanceResamplingFilter::estimate_type`

Estimation type to be provided by the filter at X vector.

7.8.5.3 Nparticles

`long int BayesianFiltering::SamplingImportanceResamplingFilter::Nparticles`

Number of particles.

7.8.5.4 Wparticles

`Eigen::VectorXd BayesianFiltering::SamplingImportanceResamplingFilter::Wparticles`

Weight for each particle, set to 1/Nparticles after resampling.

7.8.5.5 Xparticles

`Eigen::MatrixXd BayesianFiltering::SamplingImportanceResamplingFilter::Xparticles`

X particles, one per col

7.8.5.6 Yparticles

`Eigen::MatrixXd BayesianFiltering::SamplingImportanceResamplingFilter::Yparticles`

Y particles, one per col

The documentation for this class was generated from the following file:

- [bayesianfiltering.hpp](#)

7.9 BayesianFiltering::StateSpaceLinearModel Class Reference

Represents the parameters of a discrete time, generic linear state space model:

```
#include <bayesianfiltering.hpp>
```

Public Member Functions

- void [SetDimensions](#) (int _Nx, int _Nu, int _Ny, int _Nw, int _Nv)
Define dimensions of system variables and adjust size of model parameters accordingly.
- [Result ProcessModel](#) (Eigen::VectorXd &X, Eigen::VectorXd &U, Eigen::VectorXd &W)
Compute process model to update state vector X (in place).
- [Result MeasurementModel](#) (Eigen::VectorXd &Y, Eigen::VectorXd &X, Eigen::VectorXd &V)
Compute system output Y from measurement model.

Public Attributes

- Eigen::MatrixXd [A](#)
Process model transition matrix.
- Eigen::MatrixXd [B](#)
Process model input matrix.
- Eigen::MatrixXd [C](#)
Measurement model matrix.
- Eigen::MatrixXd [Q](#)
Process model noise covariance matrix.
- Eigen::MatrixXd [R](#)
Measurement model noise covariance matrix.
- Eigen::VectorXd [X0](#)
Initial state vector distribution mean vector parameter.
- Eigen::MatrixXd [P0](#)
Initial state vector distribution covariance matrix parameter.
- int [Nx](#)
Number of state variables in X vector.
- int [Nu](#)
Number of inputs in U vector.
- int [Ny](#)
Number of measurements in Y vector.
- int [Nw](#)
Number of process noise in W vector.
- int [Nv](#)
Number of measurement noise in V vector.

7.9.1 Detailed Description

Represents the parameters of a discrete time, generic linear state space model:

$$\begin{aligned}x(k) &= A \cdot x(k-1) + B \cdot u(k) + w(k) \quad \{\text{Process equation}\} \\ y(k) &= C \cdot x(k) + v(k) \quad \{\text{Measurement equation}\}\end{aligned}$$

with $x(k)$ being state variables vector, $u(k)$ measurement vector and $y(k)$ measurement vector. Also, $w(k) \sim N(0, Q)$ and $v(k) \sim N(0, R)$ are respectively process and measurement noises.

This class contains only model parameters and evaluations of process and measurement equations. In case $w(k)$ and/or $v(k)$ are dependent of other noise sources, Q and R should consider the correct propagation from noise sources covariances.

Initial state is a random variable following $x(0) \sim N(X0, P0)$

Also $\dim(x) = N_x$, $\dim(u) = N_u$, $\dim(y) = N_y$, $\dim(w) = N_w$, $\dim(v) = N_v$. These dimensions should be first established by calling `SetDimensions()`, which make resizing of system parameters. After that, since parameters are public, they can be set at any time.

7.9.2 Member Function Documentation

7.9.2.1 MeasurementModel()

```
Result BayesianFiltering::StateSpaceLinearModel::MeasurementModel (
    Eigen::VectorXd & Y,
    Eigen::VectorXd & X,
    Eigen::VectorXd & V ) [inline]
```

Compute system output Y from measurement model.

Parameters

out	Y	the computed output
in	X	state vector with $x(k)$
in	V	current sample of measurement noise

7.9.2.2 ProcessModel()

```
Result BayesianFiltering::StateSpaceLinearModel::ProcessModel (
    Eigen::VectorXd & X,
    Eigen::VectorXd & U,
    Eigen::VectorXd & W ) [inline]
```

Compute process model to update state vector X (in place).

Parameters

in, out	X	state vector with $x(k-1)$ to be updated to $x(k)$
in	U	input vector
in	W	current sample of process noise

7.9.2.3 SetDimensions()

```
void BayesianFiltering::StateSpaceLinearModel::SetDimensions (
    int _Nx,
    int _Nu,
    int _Ny,
    int _Nw,
    int _Nv ) [inline]
```

Define dimensions of system variables and adjust size of model parameters accordingly.

Parameters

in	$_Nx$	Number of state variables in X vector
in	$_Nu$	Number of inputs in U vector
in	$_Ny$	Number of measurements in Y vector
in	$_Nw$	Number of process noise in W vector
in	$_Nv$	Number of measurement noise in V vector

7.9.3 Member Data Documentation**7.9.3.1 A**

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::A
```

Process model transition matrix.

7.9.3.2 B

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::B
```

Process model input matrix.

7.9.3.3 C

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::C
```

Measurement model matrix.

7.9.3.4 Nu

```
int BayesianFiltering::StateSpaceLinearModel::Nu
```

Number of inputs in U vector.

7.9.3.5 Nv

```
int BayesianFiltering::StateSpaceLinearModel::Nv
```

Number of measurement noise in V vector.

7.9.3.6 Nw

```
int BayesianFiltering::StateSpaceLinearModel::Nw
```

Number of process noise in W vector.

7.9.3.7 Nx

```
int BayesianFiltering::StateSpaceLinearModel::Nx
```

Number of state variables in X vector.

7.9.3.8 Ny

```
int BayesianFiltering::StateSpaceLinearModel::Ny
```

Number of measurements in Y vector.

7.9.3.9 P0

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::P0
```

Initial state vector distribution covariance matrix parameter.

7.9.3.10 Q

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::Q
```

Process model noise covariance matrix.

7.9.3.11 R

```
Eigen::MatrixXd BayesianFiltering::StateSpaceLinearModel::R
```

Measurement model noise covariance matrix.

7.9.3.12 X0

```
Eigen::VectorXd BayesianFiltering::StateSpaceLinearModel::X0
```

Initial state vector distribution mean vector parameter.

The documentation for this class was generated from the following file:

- [bayesianfiltering.hpp](#)

7.10 BayesianFiltering::StateSpaceNonlinearModelBase Class Reference

Represents the parameters of a discrete time, generic nonlinear state space model:

```
#include <bayesianfiltering.hpp>
```

Public Member Functions

- virtual [~StateSpaceNonlinearModelBase](#) ()=default
- void [SetDimensions](#) (int _Nx, int _Nu, int _Ny, int _Nw, int _Nv)
Define dimensions of system variables and adjust size of model parameters accordingly.
- virtual [Result ProcessModel](#) (Eigen::VectorXd &X, Eigen::VectorXd &U, double t, Eigen::VectorXd &W)=0
Compute process model to update state vector X (in place). Virtual function to be defined in derived class.
- virtual [Result MeasurementModel](#) (Eigen::VectorXd &Y, Eigen::VectorXd &X, double t, Eigen::VectorXd &V)=0
Compute system output Y from measurement model. Virtual function to be defined in derived class.
- virtual [Result UpdatedfdX](#) (Eigen::VectorXd &X, Eigen::VectorXd &U, double t, Eigen::VectorXd &W)=0
Update dfdX internal class member, the process model derivative with respect to $x(k-1)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.
- virtual [Result UpdatedfdW](#) (Eigen::VectorXd &X, Eigen::VectorXd &U, double t, Eigen::VectorXd &W)=0
Update dfdW internal class member, the process model derivative with respect to $w(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.
- virtual [Result UpdatedhdX](#) (Eigen::VectorXd &X, double t, Eigen::VectorXd &V)=0
Update dhdX internal class member, the measurement model derivative with respect to $x(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.
- virtual [Result UpdatedhdV](#) (Eigen::VectorXd &X, double t, Eigen::VectorXd &V)=0
Update dhdV internal class member, the measurement model derivative with respect to $v(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Public Attributes

- Eigen::MatrixXd [dfdX](#)
Partial derivative of $f(X, U, W)$ with respect to X .
- Eigen::MatrixXd [dfdW](#)
Partial derivative of $f(X, U, W)$ with respect to W .
- Eigen::MatrixXd [dhdX](#)
Partial derivative of $h(X, V)$ with respect to X .
- Eigen::MatrixXd [dhdV](#)
Partial derivative of $h(X, V)$ with respect to V .
- Eigen::MatrixXd [Q](#)
Process model noise covariance matrix.
- Eigen::MatrixXd [R](#)
Measurement model noise covariance matrix.
- Eigen::VectorXd [X0](#)
Initial state vector distribution mean vector parameter.
- Eigen::MatrixXd [P0](#)
Initial state vector distribution covariance matrix parameter.
- int [Nx](#)
Number of state variables in X vector.
- int [Nu](#)
Number of inputs in U vector.
- int [Ny](#)
Number of measurements in Y vector.
- int [Nw](#)
Number of process noise in W vector.
- int [Nv](#)
Number of measurement noise in V vector.

7.10.1 Detailed Description

Represents the parameters of a discrete time, generic nonlinear state space model:

$$\begin{aligned} \mathbf{x}(k) &= \mathbf{f}(\mathbf{x}(k-1), \mathbf{u}(k), \mathbf{w}(k)) \quad \{\text{Process equation}\} \\ \mathbf{y}(k) &= \mathbf{h}(\mathbf{x}(k), \mathbf{v}(k)) \quad \{\text{Measurement equation}\} \end{aligned}$$

with $\mathbf{x}(k)$ being state variables vector, $\mathbf{u}(k)$ measurement vector and $\mathbf{y}(k)$ measurement vector. Also, $\mathbf{w}(k) \sim N(0, Q)$ and $\mathbf{v}(k) \sim N(0, R)$ are respectively process and measurement noises.

This class contains only model parameters. It is a base virtual class, with computation of process and measurement equations as virtual member functions. Its definition should be done in a derived class at user side. Partial derivatives of process and measurement models should also be defined at a derived class. Some filters use such derivatives.

Initial state is a random variable following $\mathbf{x}(0) \sim N(\mathbf{X0}, \mathbf{P0})$

Also $\dim(\mathbf{x}) = N_x$, $\dim(\mathbf{u}) = N_u$, $\dim(\mathbf{y}) = N_y$, $\dim(\mathbf{w}) = N_w$, $\dim(\mathbf{v}) = N_v$. These dimensions should be first established by calling `SetDimensions()`, which make resizing of system parameters. After that, since parameters are public, they can be set at any time.

7.10.2 Constructor & Destructor Documentation

7.10.2.1 ~StateSpaceNonlinearModelBase()

```
virtual BayesianFiltering::StateSpaceNonlinearModelBase::~~StateSpaceNonlinearModelBase ( )
[virtual], [default]
```

7.10.3 Member Function Documentation

7.10.3.1 MeasurementModel()

```
virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::MeasurementModel (
    Eigen::VectorXd & Y,
    Eigen::VectorXd & X,
    double t,
    Eigen::VectorXd & V ) [pure virtual]
```

Compute system output Y from measurement model. Virtual function to be defined in derived class.

Parameters

out	Y	the computed output
in	X	state vector with $x(k)$
in	t	time instant scalar
in	V	current sample of measurement noise

7.10.3.2 ProcessModel()

```
virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::ProcessModel (
    Eigen::VectorXd & X,
    Eigen::VectorXd & U,
    double t,
    Eigen::VectorXd & W ) [pure virtual]
```

Compute process model to update state vector X (in place). Virtual function to be defined in derived class.

Parameters

in, out	X	state vector with $x(k-1)$ to be updated to $x(k)$
in	U	input vector
in	t	time instant scalar
in	W	current sample of process noise

7.10.3.3 SetDimensions()

```
void BayesianFiltering::StateSpaceNonlinearModelBase::SetDimensions (
    int _Nx,
```

```

int _Nu,
int _Ny,
int _Nw,
int _Nv ) [inline]

```

Define dimensions of system variables and adjust size of model parameters accordingly.

Parameters

in	<code>_Nx</code>	Number of state variables in X vector
in	<code>_Nu</code>	Number of inputs in U vector
in	<code>_Ny</code>	Number of measurements in Y vector
in	<code>_Nw</code>	Number of process noise in W vector
in	<code>_Nv</code>	Number of measurement noise in V vector

7.10.3.4 UpdatedfdW()

```

virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::UpdatedfdW (
    Eigen::VectorXd & X,
    Eigen::VectorXd & U,
    double t,
    Eigen::VectorXd & W ) [pure virtual]

```

Update dfdW internal class member, the process model derivative with respect to $w(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Parameters

in	X	state vector containing $x(k-1)$
in	U	input vector
in	t	time instant scalar
in	W	current sample of process noise

7.10.3.5 UpdatedfdX()

```

virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::UpdatedfdX (
    Eigen::VectorXd & X,
    Eigen::VectorXd & U,
    double t,
    Eigen::VectorXd & W ) [pure virtual]

```

Update dfdX internal class member, the process model derivative with respect to $x(k-1)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Parameters

in	X	state vector containing $x(k-1)$
in	U	input vector
in	t	time instant scalar
in	W	current sample of process noise

7.10.3.6 UpdatedhdV()

```
virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::UpdatedhdV (
    Eigen::VectorXd & X,
    double t,
    Eigen::VectorXd & V ) [pure virtual]
```

Update dhdV internal class member, the measurement model derivative with respect to $v(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Parameters

in	X	state vector containing $x(k-1)$
in	t	time instant scalar
in	V	current sample of measurement noise

7.10.3.7 UpdatedhdX()

```
virtual Result BayesianFiltering::StateSpaceNonlinearModelBase::UpdatedhdX (
    Eigen::VectorXd & X,
    double t,
    Eigen::VectorXd & V ) [pure virtual]
```

Update dhdX internal class member, the measurement model derivative with respect to $x(k)$. Necessary only in filters using system matrix derivatives, where this function is called from inside the filter class. Virtual function to be defined in derived class.

Parameters

in	X	state vector containing $x(k-1)$
in	t	time instant scalar
in	V	current sample of measurement noise

7.10.4 Member Data Documentation

7.10.4.1 dfdW

```
Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::dfdW
```

Partial derivative of $f(X,U,W)$ with respect to W .

7.10.4.2 dfdX

```
Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::dfdX
```

Partial derivative of $f(X,U,W)$ with respect to X .

7.10.4.3 dhdV

`Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::dhdV`

Partial derivative of $h(X,V)$ with respect to V .

7.10.4.4 dhdX

`Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::dhdX`

Partial derivative of $h(X,V)$ with respect to X .

7.10.4.5 Nu

`int BayesianFiltering::StateSpaceNonlinearModelBase::Nu`

Number of inputs in U vector.

7.10.4.6 Nv

`int BayesianFiltering::StateSpaceNonlinearModelBase::Nv`

Number of measurement noise in V vector.

7.10.4.7 Nw

`int BayesianFiltering::StateSpaceNonlinearModelBase::Nw`

Number of process noise in W vector.

7.10.4.8 Nx

`int BayesianFiltering::StateSpaceNonlinearModelBase::Nx`

Number of state variables in X vector.

7.10.4.9 Ny

`int BayesianFiltering::StateSpaceNonlinearModelBase::Ny`

Number of measurements in Y vector.

7.10.4.10 P0

`Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::P0`

Initial state vector distribution covariance matrix parameter.

7.10.4.11 Q

`Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::Q`

Process model noise covariance matrix.

7.10.4.12 R

`Eigen::MatrixXd BayesianFiltering::StateSpaceNonlinearModelBase::R`

Measurement model noise covariance matrix.

7.10.4.13 X0

`Eigen::VectorXd BayesianFiltering::StateSpaceNonlinearModelBase::X0`

Initial state vector distribution mean vector parameter.

The documentation for this class was generated from the following file:

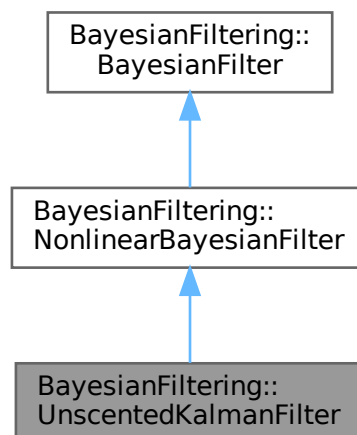
- [bayesianfiltering.hpp](#)

7.11 BayesianFiltering::UnscentedKalmanFilter Class Reference

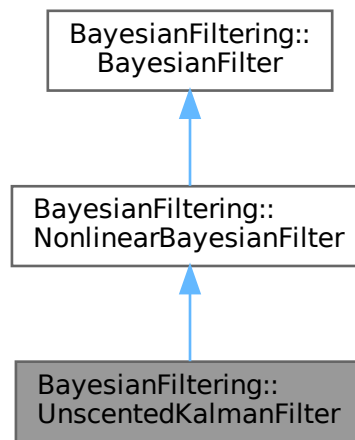
Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models.

```
#include <bayesianfiltering.hpp>
```

Inheritance diagram for BayesianFiltering::UnscentedKalmanFilter:



Collaboration diagram for BayesianFiltering::UnscentedKalmanFilter:



Public Member Functions

- virtual [~UnscentedKalmanFilter](#) ()
- void [Reset](#) ([StateSpaceNonlinearModelBase](#) &nonlinear_model) override
Reset filter, first procedure before prediction/correction.
- [Result Prediction](#) ([Eigen::VectorXd](#) &U, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override
Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `nonlinear_model`. This procedure calls methods of the derived class object `nonlinear_model` in order to update model parameters. So there is no need the user to do that.
- [Result Correction](#) ([Eigen::VectorXd](#) &Y, double t, [StateSpaceNonlinearModelBase](#) &nonlinear_model) override
Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model `linear_model`. In case of time varying systems, `linear_model` class members should be updated before calling this procedure.

Public Member Functions inherited from [BayesianFiltering::NonlinearBayesianFilter](#)

- virtual [~NonlinearBayesianFilter](#) ()

Public Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- [BayesianFilter](#) ()
- virtual [~BayesianFilter](#) ()

Additional Inherited Members

Public Attributes inherited from [BayesianFiltering::BayesianFilter](#)

- Eigen::VectorXd [X](#)
State vector predictor $\hat{x}(k|k-1)$ or estimate $\hat{x}(k|k)$, depending on which stage the filter is.
- Eigen::MatrixXd [P](#)
State vector error covariance matrix predictor $P(k|k-1)$ or $P(k|k)$, depending on which stage the filter is.
- Eigen::VectorXd [Y_predicted](#)
Predicted measurement $\hat{y}(k|k-1)$.
- Eigen::VectorXd [V_predicted](#)
Predicted innovation $y(k) - \hat{y}(k|k-1)$.
- Eigen::MatrixXd [S_predicted](#)
Predicted innovation covariance matrix.
- double [d2_predicted](#)
Predicted innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.
- Eigen::VectorXd [Y_corrected](#)
Corrected measurement $\hat{y}(k|k)$.
- Eigen::VectorXd [V_corrected](#)
Corrected innovation $y(k) - \hat{y}(k|k)$.
- Eigen::MatrixXd [S_corrected](#)
Corrected innovation covariance matrix.
- double [d2_corrected](#)
Corrected innovation Mahalanobis distance, allowing to verify a filter as optimistic or pessimistic, and divergence.

Protected Member Functions inherited from [BayesianFiltering::BayesianFilter](#)

- void [Reset](#) (Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
Reset filter, first procedure before prediction/correction.

7.11.1 Detailed Description

Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models.

This implementation is specific for discrete time, continuous state models.

7.11.2 Constructor & Destructor Documentation

7.11.2.1 [~UnscentedKalmanFilter\(\)](#)

```
virtual BayesianFiltering::UnscentedKalmanFilter::~~UnscentedKalmanFilter ( ) [inline], [virtual]
```

7.11.3 Member Function Documentation

7.11.3.1 Correction()

Result BayesianFiltering::UnscentedKalmanFilter::Correction (
 Eigen::VectorXd & Y,
 double t,
 StateSpaceNonlinearModelBase & nonlinear_model) [inline], [override], [virtual]

Correction procedure, which will update (in place) the internal member variables X and P based on measurement vector Y and model linear_model. In case of time varying systems, linear_model class members should be updated before calling this procedure.

Parameters

in	Y	measurement vector of the linear system process model
in	t	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.11.3.2 Prediction()

```
Result BayesianFiltering::UnscentedKalmanFilter::Prediction (
    Eigen::VectorXd & U,
    double t,
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Prediction procedure, which will update (in place) the internal member variables X and P based on input vector U and model `nonlinear_model`. This procedure calls methods of the derived class object `nonlinear_model` in order to update model parameters. So there is no need the user to do that.

Parameters

in	U	input vector of the linear system process model
in	t	time variable, for time varying systems
in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run. Model updating and parameter calculations implemented by the user in the StateSpaceNonlinearModelBase derived object are called from inside this method.

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

7.11.3.3 Reset()

```
void BayesianFiltering::UnscentedKalmanFilter::Reset (
    StateSpaceNonlinearModelBase & nonlinear_model ) [inline], [override], [virtual]
```

Reset filter, first procedure before prediction/correction.

Parameters

in	<i>nonlinear_model</i>	StateSpaceNonlinearModelBase derived object for which the filter will run
----	------------------------	---

Implements [BayesianFiltering::NonlinearBayesianFilter](#).

The documentation for this class was generated from the following file:

- [bayesianfiltering.hpp](#)

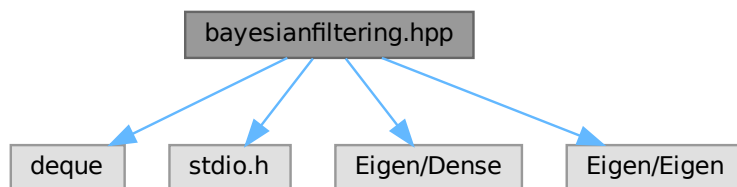
Chapter 8

File Documentation

8.1 bayesianfiltering.hpp File Reference

```
#include <deque>
#include <stdio.h>
#include <Eigen/Dense>
#include <Eigen/Eigen>
```

Include dependency graph for bayesianfiltering.hpp:



Classes

- class [BayesianFiltering::StateSpaceLinearModel](#)
Represents the parameters of a discrete time, generic linear state space model:
- class [BayesianFiltering::StateSpaceNonlinearModelBase](#)
Represents the parameters of a discrete time, generic nonlinear state space model:
- class [BayesianFiltering::BayesianFilter](#)
Base class of minimum variables of a Bayesian stochastic filter. All derived classes present filters for linear and nonlinear models. All filters follow the following loop.
- class [BayesianFiltering::LinearBayesianFilter](#)
Base class of Bayesian filters for state space linear models. See derived classes.
- class [BayesianFiltering::KalmanFilter](#)
Kalman filter, the best optimal unbiased estimator for stochastic linear state space models.

- class [BayesianFiltering::AdaptiveKalmanFilter](#)
Adaptive Kalman filter, with Q and R matrices changing when every measurement comes according two user supplied learning parameters.
- class [BayesianFiltering::RobustSelfAdaptiveKalmanFilter](#)
Robust Self Adaptive Kalman filter, with Q and R matrices changing when every measurement comes using covariance matching strategy.
- class [BayesianFiltering::NonlinearBayesianFilter](#)
Base class of Bayesian filters for state space nonlinear models. See derived classes.
- class [BayesianFiltering::ExtendedKalmanFilter](#)
Extended Kalman filter, the first order approximation-based estimator for stochastic nonlinear state space models.
- class [BayesianFiltering::UnscentedKalmanFilter](#)
Unscented Kalman filter, the Unscented Transform (UT)-based estimator for stochastic nonlinear state space models.
- class [BayesianFiltering::SamplingImportanceResamplingFilter](#)
Sampling Importance Resampling (SIR) filter, particle-based estimator for stochastic nonlinear state space models.

Namespaces

- namespace [BayesianFiltering](#)

Macros

- `#define PRINT_LINE_DEBUG {printf("\n[%s] line %i", __FILE__, __LINE__); fflush(stdout);}`
Print filename followed by line number at command line. Used commonly for debugging allowing to identify the code responsible for errors.
- `#define PRINT_EIGEN_MATRIX(X) PrintEigenMatrixXd(X,#X);`
Print pretty matrix X with the same variable name at command line (printf). Guaranteed print all lines before leaving (fflush).
- `#define PRINT_EIGEN_VECTOR(X) PrintEigenVectorXd(X,#X);`
Print pretty vector X with the same variable name at command line (printf). Guaranteed print all lines before leaving (fflush).

Typedefs

- `typedef std::pair< bool, std::string > BayesianFiltering::Result`
Encapsulated `<bool,std::string>` pair response of a function, with .first specifying status (success of failure) and .second the reason string in case of failure.

Functions

- `void PrintEigenMatrixXd (Eigen::MatrixXd &X, const char *name)`
Print pretty matrix X with name which may me different of "X" at command line (printf). Guaranteed print all lines before leaving (fflush).
- `void PrintEigenVectorXd (Eigen::VectorXd &X, const char *name)`
Print pretty vector X with name which may me different of "X" at command line (printf). Guaranteed print all lines before leaving (fflush).
- `Result BayesianFiltering::ResultFormat (bool success, const std::string &message)`
Format `<bool,std::string>` pair response with defined status and reason string.
- `template<typename ... Args>`
`Result BayesianFiltering::ResultFormat (bool success, const std::string &format, Args ... args)`
Format `<bool,std::string>` pair response with defined status and reason string to be printf-like formatted.

- [Result BayesianFiltering::MultivariateZeroMeanGaussianSampler](#) (Eigen::VectorXd &sample, Eigen::MatrixXd &covariance_matrix)
Generate multivariate $sample \sim N(0, covariance_matrix)$.
- [Result BayesianFiltering::MultivariateGaussianSampler](#) (Eigen::VectorXd &sample, Eigen::VectorXd &mean_vector, Eigen::MatrixXd &covariance_matrix)
Generate multivariate $sample \sim N(mean_vector, covariance_matrix)$.
- double [BayesianFiltering::UnivariateGaussianSampler](#) (double mean, double variance)
Generate univariate $sample \sim N(mean, variance)$.
- double [BayesianFiltering::UnivariateUniformSampler](#) (double min_value, double max_value)
Generate univariate $sample \sim U(min_value, max_value)$.

8.1.1 Macro Definition Documentation

8.1.1.1 PRINT_EIGEN_MATRIX

```
#define PRINT_EIGEN_MATRIX(  
    X ) PrintEigenMatrixXd(X, #X);
```

Print pretty matrix X with the same variable name at command line (printf). Guaranteed print all lines before leaving (fflush).

Parameters

in	X	The EIGEN::MatrixXd variable.
----	---	-------------------------------

8.1.1.2 PRINT_EIGEN_VECTOR

```
#define PRINT_EIGEN_VECTOR(  
    X ) PrintEigenVectorXd(X, #X);
```

Print pretty vector X with the same variable name at command line (printf). Guaranteed print all lines before leaving (fflush).

Parameters

in	X	The EIGEN::VectorXd variable.
----	---	-------------------------------

8.1.1.3 PRINT_LINE_DEBUG

```
#define PRINT_LINE_DEBUG {printf("\n[%s] line %i", __FILE__, __LINE__); fflush(stdout);}
```

Print filename followed by line number at command line. Used commonly for debugging allowing to identify the code responsible for errors.

Parameters

in	X	The EIGEN::VectorXd variable.
----	---	-------------------------------

8.1.2 Function Documentation

8.1.2.1 PrintEigenMatrixXd()

```
void PrintEigenMatrixXd (
    Eigen::MatrixXd & X,
    const char * name ) [inline]
```

Print pretty matrix X with name which may be different of "X" at command line (printf). Guaranteed print all lines before leaving (fflush).

Parameters

in	<i>X</i>	The EIGEN::MatrixXd variable.
in	<i>name</i>	Name of the variable.

8.1.2.2 PrintEigenVectorXd()

```
void PrintEigenVectorXd (
    Eigen::VectorXd & X,
    const char * name ) [inline]
```

Print pretty vector X with name which may be different of "X" at command line (printf). Guaranteed print all lines before leaving (fflush).

Parameters

in	<i>X</i>	The EIGEN::VectorXd variable.
in	<i>name</i>	Name of the variable.

8.2 bayesianfiltering.hpp

[Go to the documentation of this file.](#)

```
00001
00002 /*****
00003 *****/
00004 *** File:      BayesianFiltering.hpp
00005 *** Author:    Geovany A. Borges
00006 *** Contents:  BayesianFiltering header file.
00007 *****/
00008 *****/
00009 BayesianFiltering.hpp is free software: you can redistribute it and/or modify
00010 it under the terms of the GNU Lesser General Public License (LGPL) as published by
00011 the Free Software Foundation, either version 3 of the License, or any later version.
00012
00013 BayesianFiltering.hpp is distributed in the hope that it will be useful,
00014 but WITHOUT ANY WARRANTY; without even the implied warranty of
00015 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00016 GNU Lesser General Public License for more details.
00017
00018 You should have received a copy of the GNU Lesser General Public License
00019 along with BayesianFiltering.hpp. If not, see <http://www.gnu.org/licenses/>.
00020
00021 Copyright 2020 Geovany Araujo Borges
00022 *****/
00023
00024 //*****
00025 /*** Classe BayesianFiltering
```

```

00026 //*****
00027 #include <deque>
00028 #include <stdio.h>
00029
00030 #include <Eigen/Dense>
00031 #include <Eigen/Eigen>
00032
00033 #ifndef BayesianFiltering_H
00034 #define BayesianFiltering_H
00035
00036 /*****
00037  * Auxiliary Macros:
00038  *****/
00043 #define PRINT_LINE_DEBUG {printf("\n[%s] line %i", __FILE__, __LINE__); fflush(stdout);}
00044
00050 inline void PrintEigenMatrixXd(Eigen::MatrixXd &X, const char* name)
00051 {
00052     printf("\nMatrix %s (%li x %li):", name, X.rows(), X.cols());
00053     for (long int i=0; i < X.rows(); ++i){
00054         printf("\n    ");
00055         for (long int j=0; j < X.cols(); ++j){
00056             printf("%s(%li,%li)=%.10f; ", name, i, j, X(i, j));
00057         }
00058     }
00059     fflush(stdout);
00060 }
00061
00066 #define PRINT_EIGEN_MATRIX(X) PrintEigenMatrixXd(X, #X);
00067
00073 inline void PrintEigenVectorXd(Eigen::VectorXd &X, const char* name)
00074 {
00075     printf("\nVector %s (%li x %li):", name, X.rows(), X.cols());
00076     for (long int i=0; i < X.rows(); ++i){
00077         printf("\n    ");
00078         for (long int j=0; j < X.cols(); ++j){
00079             printf("%s(%li,%li)=%.10f; ", name, i, j, X(i, j));
00080         }
00081     }
00082     fflush(stdout);
00083 }
00088 #define PRINT_EIGEN_VECTOR(X) PrintEigenVectorXd(X, #X);
00089
00090 namespace BayesianFiltering {
00091
00092     /*****
00093     * Support types and functions:
00094     *****/
00099     typedef std::pair<bool, std::string> Result;
00100
00106     inline Result ResultFormat(bool success, const std::string& message) {
00107         return std::make_pair(success, message);
00108     }
00109
00116     template<typename ... Args>
00117     Result ResultFormat(bool success, const std::string& format, Args ... args )
00118     {
00119         Result res; res.first = success;
00120         int size_s = std::snprintf( nullptr, 0, format.c_str(), args ... ) + 1; // Extra space for
00121         '\0'
00122         if ( size_s <= 0 ) { throw std::runtime_error( "Error during formatting." ); }
00123         auto size = static_cast<size_t>( size_s );
00124         std::unique_ptr<char[]> buf( new char[ size ] );
00125         std::snprintf( buf.get(), size, format.c_str(), args ... );
00126         res.second = std::string( buf.get(), buf.get() + size - 1 ); // We don't want the '\0' inside
00127         return res;
00128     };
00129
00136     Result MultivariateZeroMeanGaussianSampler(Eigen::VectorXd & sample, Eigen::MatrixXd &
covariance_matrix)
00137     {
00138         std::random_device rd;
00139         std::mt19937 gen(rd());
00140         std::normal_distribution<double> dist(0.0, 1.0);
00141
00142         // Verify dimensions
00143         if (covariance_matrix.rows() != covariance_matrix.cols()) return ResultFormat(false,
"covariance_matrix should be square (%li x %li)", covariance_matrix.rows(), covariance_matrix.cols());
00144
00145         // Compute Cholesky Decomposition (LLT)
00146         Eigen::LLT<Eigen::MatrixXd> llt(covariance_matrix);
00147         Eigen::MatrixXd L = llt.matrixL();
00148
00149         // Generate independent standard normal vector Z
00150         Eigen::VectorXd Z = Eigen::VectorXd::NullaryExpr(covariance_matrix.rows(), [&]() { return
dist(gen); });

```

```

00151
00152     // Transform to Multivariate Normal
00153     sample = L * Z;
00154
00155     return ResultFormat(true, "");
00156 }
00157
00165 Result MultivariateGaussianSampler(Eigen::VectorXd & sample, Eigen::VectorXd & mean_vector,
Eigen::MatrixXd & covariance_matrix)
00166 {
00167     // Verify dimensions
00168     if (mean_vector.rows() != covariance_matrix.rows()) return ResultFormat(false, "mean_vector
number of lines (%li) different of covariance_matrix number of lines
(%li)", mean_vector.rows(), covariance_matrix.rows());
00169     // Covariance Matrix will be checked at zero mean Gaussian sampler
00170
00171     // Generate sample from zero mean sample:
00172     Result res = MultivariateZeroMeanGaussianSampler(sample, covariance_matrix);
00173     if (!res.first) return res;
00174
00175     sample += mean_vector;
00176
00177     return res;
00178 }
00179
00186 double UnivariateGaussianSampler(double mean, double variance)
00187 {
00188     std::random_device rd;
00189     std::mt19937 gen(rd());
00190     std::normal_distribution<double> dist(0.0, 1.0);
00191
00192     return mean + sqrt(variance) * dist(gen);
00193 }
00194
00201 double UnivariateUniformSampler(double min_value, double max_value)
00202 {
00203     std::random_device rd;
00204     std::mt19937 gen(rd());
00205     std::uniform_real_distribution<double> dist(0.0, 1.0);
00206
00207     return dist(gen);
00208 }
00209
00210 /*****
00211  * Linear model class:
00212  *****/
00229 class StateSpaceLinearModel
00230 {
00231 public:
00232     Eigen::MatrixXd A;
00233     Eigen::MatrixXd B;
00234     Eigen::MatrixXd C;
00235     Eigen::MatrixXd Q;
00236     Eigen::MatrixXd R;
00237     Eigen::VectorXd X0;
00238     Eigen::MatrixXd P0;
00239     int Nx;
00240     int Nu;
00241     int Ny;
00242     int Nw;
00243     int Nv;
00244
00253 void SetDimensions(int _Nx, int _Nu, int _Ny, int _Nw, int _Nv)
00254 {
00255     Nx = _Nx; Nu = _Nu; Ny = _Ny; Nw = _Nw; Nv = _Nv;
00256     A.resize(Nx, Nx);
00257     B.resize(Nx, Nu);
00258     C.resize(Ny, Nx);
00259     Q.resize(Nx, Nx);
00260     R.resize(Ny, Ny);
00261     X0.resize(Nx);
00262     P0.resize(Nx, Nx);
00263 }
00264
00271 Result ProcessModel(Eigen::VectorXd & X, Eigen::VectorXd & U, Eigen::VectorXd & W)
00272 {
00273     Eigen::VectorXd Xpos;
00274     Xpos = A * X + B * U + W; // By using an auxiliary variable, it avoids any risk of copy in
place
00275     X = Xpos;
00276     return BayesianFiltering::ResultFormat(true, "");
00277 }
00278
00285 Result MeasurementModel(Eigen::VectorXd & Y, Eigen::VectorXd & X, Eigen::VectorXd & V)
00286 {
00287     Y = C * X + V;
00288     return BayesianFiltering::ResultFormat(true, "");

```

```

00289     }
00290
00291 };
00292
00293 /*****
00294  * Nonlinear model base class:
00295  *****/
00313 class StateSpaceNonlinearModelBase
00314 {
00315 public:
00316     Eigen::MatrixXd dfdX;
00317     Eigen::MatrixXd dfdW;
00318     Eigen::MatrixXd dhdX;
00319     Eigen::MatrixXd dhdV;
00320     Eigen::MatrixXd Q;
00321     Eigen::MatrixXd R;
00322     Eigen::VectorXd X0;
00323     Eigen::MatrixXd P0;
00324     int Nx;
00325     int Nu;
00326     int Ny;
00327     int Nw;
00328     int Nv;
00329
00330     // Always provide a virtual destructor if your class has virtual functions!
00331     virtual ~StateSpaceNonlinearModelBase() = default;
00332
00341     void SetDimensions(int _Nx, int _Nu, int _Ny, int _Nw, int _Nv)
00342     {
00343         Nx = _Nx; Nu = _Nu; Ny = _Ny; Nw = _Nw; Nv = _Nv;
00344         dfdX.resize(Nx, Nx);
00345         dfdW.resize(Nx, Nw);
00346         dhdX.resize(Ny, Nx);
00347         dhdV.resize(Ny, Nv);
00348         Q.resize(Nw, Nw);
00349         R.resize(Nv, Nv);
00350         X0.resize(Nx);
00351         P0.resize(Nx, Nx);
00352     }
00353
00361     virtual Result ProcessModel(Eigen::VectorXd & X, Eigen::VectorXd & U, double t,
Eigen::VectorXd & W)=0; // Deve ser definido pelo usuario
00369     virtual Result MeasurementModel(Eigen::VectorXd & Y, Eigen::VectorXd & X, double t,
Eigen::VectorXd & V)=0; // Deve ser definido pelo usuario
00377     virtual Result UpdatedfdX(Eigen::VectorXd & X, Eigen::VectorXd & U, double t, Eigen::VectorXd
& W)=0; // Deve ser definido pelo usuario
00385     virtual Result UpdatedfdW(Eigen::VectorXd & X, Eigen::VectorXd & U, double t, Eigen::VectorXd
& W)=0; // Deve ser definido pelo usuario
00392     virtual Result UpdateddhdX(Eigen::VectorXd & X, double t, Eigen::VectorXd & V)=0; // Deve ser
definido pelo usuario
00399     virtual Result UpdateddhdV(Eigen::VectorXd & X, double t, Eigen::VectorXd & V)=0; // Deve ser
definido pelo usuario
00400 };
00401
00402 /*****
00403  * Bayesian filter base class:
00404  *****/
00421 class BayesianFilter
00422 {
00423 public:
00424     Eigen::VectorXd X;
00425     Eigen::MatrixXd P;
00426
00427     Eigen::VectorXd Y_predicted;
00428     Eigen::VectorXd V_predicted;
00429     Eigen::MatrixXd S_predicted;
00430     double d2_predicted;
00431
00432     Eigen::VectorXd Y_corrected;
00433     Eigen::VectorXd V_corrected;
00434     Eigen::MatrixXd S_corrected;
00435     double d2_corrected;
00436
00437     BayesianFilter()
00438     {
00439         d2_predicted = 0;
00440         d2_corrected = 0;
00441     }
00442     virtual ~BayesianFilter(){}
00443
00444     protected: // Only derived classes have access
00445
00453     void Reset(Eigen::VectorXd &X0, Eigen::MatrixXd &P0, int Nx, int Ny)
00454     {
00455
00456         X.resize(Nx, 1);
00457         P.resize(Nx, Nx);

```

```

00458         Y_predicted.resize(Ny, 1);
00459         V_predicted.resize(Ny, 1);
00460         S_predicted.resize(Ny, Ny);
00461         Y_corrected.resize(Ny, 1);
00462         V_corrected.resize(Ny, 1);
00463         S_corrected.resize(Ny, Ny);
00464
00465         d2_predicted = 0;
00466         d2_corrected = 0;
00467         X = X0;
00468         P = P0;
00469     }
00470 };
00471
00472 class LinearBayesianFilter : public BayesianFilter
00473 {
00474 public:
00475     virtual ~LinearBayesianFilter() {std::cout << "\nLinearBayesianFilter Base destroyed\n";}
00476
00477     virtual Result Prediction(Eigen::VectorXd & U, StateSpaceLinearModel& linear_model)=0;
00478     virtual Result Correction(Eigen::VectorXd & Y, StateSpaceLinearModel& linear_model)=0;
00479     virtual void Reset(StateSpaceLinearModel& linear_model)=0;
00480
00481 protected:
00482     Result CheckModel(StateSpaceLinearModel& linear_model)
00483     {
00484         if (linear_model.A.rows() != X.rows()) return ResultFormat(false, "Number of rows of A should
be the same number of rows of X");
00485         if (linear_model.A.cols() != X.rows()) return ResultFormat(false, "Number of cols of A should
be the same number of rows of X");
00486         if (linear_model.B.rows() != X.rows()) return ResultFormat(false, "Number of rows of B should
be the same number of rows of X");
00487         return ResultFormat(true, "");
00488     }
00489 };
00490
00491 class KalmanFilter : public LinearBayesianFilter
00492 {
00493 public:
00494     virtual ~KalmanFilter() {std::cout << "\nKalmanFilter derived class destroyed\n";}
00495
00496     void Reset(StateSpaceLinearModel& linear_model) override
00497     {
00498         BayesianFilter::Reset(linear_model.X0, linear_model.P0, linear_model.Nx, linear_model.Ny);
00499     }
00500
00501     Result Prediction(Eigen::VectorXd & U, StateSpaceLinearModel& linear_model) override
00502     {
00503         Result res = CheckModel(linear_model); if(!res.first) return res;
00504
00505         // Predicao
00506         X = linear_model.A * X + linear_model.B * U;
00507         P = linear_model.A * P * linear_model.A.transpose() + linear_model.Q;
00508         return ResultFormat(true, "");
00509     }
00510
00511     Result Correction(Eigen::VectorXd & Y, StateSpaceLinearModel& linear_model) override
00512     {
00513         Result res = CheckModel(linear_model); if(!res.first) return res;
00514
00515         // Correcao
00516         Y_predicted = linear_model.C * X;
00517         V_predicted = Y - Y_predicted;
00518         S_predicted = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00519         d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
00520
00521         Eigen::MatrixXd G = P * linear_model.C.transpose() * S_predicted.inverse();
00522         X = X + G * V_predicted;
00523         //P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P;
00524         // More numerically stable P update:
00525         P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P
00526         *((Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C).transpose() +
G*linear_model.R)*G.transpose());
00527
00528         Y_corrected = linear_model.C * X;
00529         V_corrected = Y - Y_corrected;
00530         S_corrected = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00531         d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
00532
00533         return ResultFormat(true, "");
00534     }
00535 };
00536
00537 class AdaptiveKalmanFilter : public LinearBayesianFilter
00538 {
00539 public:
00540     double alpha;

```

```

00567     double beta;
00568
00569     virtual ~AdaptiveKalmanFilter() {std::cout << "\nAdaptiveKalmanFilter derived class
destroyed\n";}
00570
00571     void Reset(StateSpaceLinearModel& linear_model, double _alpha, double _beta)
00572     {
00573         alpha = _alpha;
00574         beta = _beta;
00575         BayesianFilter::Reset(linear_model.X0, linear_model.P0, linear_model.Nx, linear_model.Ny);
00576     }
00577
00578     void Reset(StateSpaceLinearModel& linear_model) override
00579     {
00580         Reset(linear_model, 0.95, 0.99);
00581     }
00582
00583     Result Prediction(Eigen::VectorXd & U, StateSpaceLinearModel& linear_model) override
00584     {
00585         Result res = CheckModel(linear_model); if(!res.first) return res;
00586         // Predicao
00587         X = linear_model.A * X + linear_model.B * U;
00588         P = linear_model.A * P * linear_model.A.transpose() + linear_model.Q;
00589         return ResultFormat(true, "");
00590     }
00591
00592     Result Correction(Eigen::VectorXd & Y, StateSpaceLinearModel& linear_model) override
00593     {
00594         Result res = CheckModel(linear_model); if(!res.first) return res;
00595
00596         // Innovation
00597         Y_predicted = linear_model.C * X;
00598         V_predicted = Y - Y_predicted;
00599         S_predicted = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00600         d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
00601         Eigen::MatrixXd P_predicted = P;
00602
00603         // State update
00604         Eigen::MatrixXd G = P * linear_model.C.transpose() * S_predicted.inverse();
00605         X = X + G * V_predicted;
00606         //P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P;
00607         // More numerically stable P update:
00608         P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P
00609         * ((Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C).transpose() +
G*linear_model.R*G.transpose());
00610
00611         // Innovation corrected
00612         Y_corrected = linear_model.C * X;
00613         V_corrected = Y - Y_corrected;
00614         S_corrected = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00615         d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
00616
00617         // R and Q updates
00618         linear_model.R = alpha * linear_model.R + (1.0-alpha)*(V_predicted *
V_predicted.transpose() + linear_model.C * P_predicted * linear_model.C.transpose());
00619         linear_model.Q = beta * linear_model.Q + (1.0-beta)*(G * V_predicted *
V_predicted.transpose() * G.transpose());
00620
00621         return ResultFormat(true, "");
00622     }
00623 };
00624
00625 class RobustSelfAdaptiveKalmanFilter : public LinearBayesianFilter
00626 {
00627     Eigen::MatrixXd P_corrected;
00628
00629 public:
00630
00631     virtual ~RobustSelfAdaptiveKalmanFilter() {std::cout << "\nRobustSelfAdaptiveKalmanFilter
derived class destroyed\n";}
00632
00633     void Reset(StateSpaceLinearModel& linear_model) override
00634     {
00635         BayesianFilter::Reset(linear_model.X0, linear_model.P0, linear_model.Nx, linear_model.Ny);
00636         P_corrected = linear_model.P0;
00637     }
00638
00639     Result Prediction(Eigen::VectorXd & U, StateSpaceLinearModel& linear_model) override
00640     {
00641         Result res = CheckModel(linear_model); if(!res.first) return res;
00642         // Predicao
00643         X = linear_model.A * X + linear_model.B * U;
00644         P = linear_model.A * P * linear_model.A.transpose() + linear_model.Q;
00645         return ResultFormat(true, "");
00646     }
00647 }

```

```

00684
00690     Result Correction(Eigen::VectorXd & Y, StateSpaceLinearModel& linear_model) override
00691     {
00692         Result res = CheckModel(linear_model); if(!res.first) return res;
00693
00694         // Ohmega calculation
00695         double sigma = linear_model.R(0,0);
00696         double epsilon = abs(V_corrected(0,0));
00697         double ohmega = 0.0;
00698         if (sigma > 0.0){
00699             ohmega = (epsilon <= sigma) ? 1.0 : sigma/epsilon;
00700         }
00701
00702         // R update
00703         linear_model.R = ohmega * linear_model.R + (1-ohmega)*(V_corrected *
V_corrected.transpose() + linear_model.C * P_corrected * linear_model.C.transpose());
00704
00705         // Innovation
00706         Y_predicted = linear_model.C * X;
00707         V_predicted = Y - Y_predicted;
00708         S_predicted = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00709         d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
00710
00711         // State update
00712         Eigen::MatrixXd G = P * linear_model.C.transpose() * S_predicted.inverse();
00713         X = X + G * V_predicted;
00714         //P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P;
00715         // More numerically stable P update:
00716         P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C) * P
*((Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * linear_model.C).transpose() +
G*linear_model.R*G.transpose());
00717
00718         // Innovation corrected
00719         Y_corrected = linear_model.C * X;
00720         V_corrected = Y - Y_corrected;
00721         S_corrected = linear_model.C * P * linear_model.C.transpose() + linear_model.R;
00722         d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
00723
00724         // Q update
00725         linear_model.Q = ohmega * linear_model.Q + (1-ohmega)*(G * V_corrected *
V_corrected.transpose() * G.transpose());
00726
00727         // P corrected, to be used in next interaction
00728         P_corrected = P;
00729
00730         return ResultFormat(true, "");
00731     }
00732 };
00733
00734 class NonlinearBayesianFilter : public BayesianFilter
00735 {
00736 public:
00737     virtual ~NonlinearBayesianFilter() {std::cout << "\nNonlinearBayesianFilter Base destroyed\n";}
00738
00739     virtual Result Prediction(Eigen::VectorXd & U, double t, StateSpaceNonlinearModelBase&
nonlinear_model)=0;
00740     virtual Result Correction(Eigen::VectorXd & Y, double t, StateSpaceNonlinearModelBase&
nonlinear_model)=0;
00741     virtual void Reset(StateSpaceNonlinearModelBase& nonlinear_model)=0;
00742 };
00743
00744 class ExtendedKalmanFilter : public NonlinearBayesianFilter
00745 {
00746 public:
00747     virtual ~ExtendedKalmanFilter() {std::cout << "\nExtendedKalmanFilter derived class
destroyed\n";}
00748
00749     void Reset(StateSpaceNonlinearModelBase& nonlinear_model) override
00750     {
00751         BayesianFilter::Reset(nonlinear_model.X0, nonlinear_model.P0, nonlinear_model.Nx,
nonlinear_model.Ny);
00752     }
00753
00754     Result Prediction(Eigen::VectorXd & U, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
00755     {
00756         // Update model (linearized on previous correction state)
00757         Eigen::VectorXd W = Eigen::MatrixXd::Zero(nonlinear_model.dfdW.rows(), 1);
00758         nonlinear_model.UpdatedfdX(X,U,t,W);
00759         nonlinear_model.UpdatedfdW(X,U,t,W);
00760
00761         // Predicao
00762         Result res = nonlinear_model.ProcessModel(X,U,t,W);
00763         if(!res.first) return res;
00764         P = nonlinear_model.dfdX * P * nonlinear_model.dfdX.transpose() +
nonlinear_model.dfdW*nonlinear_model.Q*nonlinear_model.dfdW.transpose();

```

```

00784
00785         return res;
00786     }
00787
00794     Result Correction(Eigen::VectorXd & Y, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
00795     {
00796         Result res;
00797
00798         // Update model (linearized on predicted state)
00799
00800         Eigen::VectorXd V = Eigen::VectorXd::Zero(nonlinear_model.dhdV.rows(), 1);
00801         nonlinear_model.UpdatedhdX(X,t,V);
00802         nonlinear_model.UpdatedhdV(X,t,V);
00803
00804         // State correction:
00805         res = nonlinear_model.MeasurementModel(Y_predicted,X,t,V);
00806         if(!res.first) return res;
00807
00808         V_predicted = Y - Y_predicted;
00809         S_predicted = nonlinear_model.dhdX * P * nonlinear_model.dhdX.transpose() +
nonlinear_model.dhdV * nonlinear_model.R * nonlinear_model.dhdV.transpose();
00810         d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
00811
00812         Eigen::MatrixXd G = P * nonlinear_model.dhdX.transpose() * S_predicted.inverse();
00813         X = X + G * V_predicted;
00814         // P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * nonlinear_model.dhdX) * P;
00815         // More numerically stable P update:
00816         P = (Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * nonlinear_model.dhdX) * P
* ((Eigen::MatrixXd::Identity(X.rows(), X.rows()) - G * nonlinear_model.dhdX).transpose() +
G*nonlinear_model.R*G.transpose());
00817
00818         // Update model (linearized on corrected state)
00819         V = Eigen::VectorXd::Zero(nonlinear_model.dhdV.rows(), 1);
00820         nonlinear_model.UpdatedhdX(X,t,V);
00821         nonlinear_model.UpdatedhdV(X,t,V);
00822
00823         // Generate corrected output, innovation and d^2:
00824         res = nonlinear_model.MeasurementModel(Y_corrected,X,t,V);
00825         if(!res.first) return res;
00826         V_corrected = Y - Y_corrected;
00827         S_corrected = nonlinear_model.dhdX * P * nonlinear_model.dhdX.transpose() +
nonlinear_model.dhdV * nonlinear_model.R * nonlinear_model.dhdV.transpose();
00828         d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
00829
00830         return res;
00831     }
00832 }
00833 };
00834
00840 class UnscentedKalmanFilter : public NonlinearBayesianFilter
00841 {
00842 private:
00843     void InitializeWeights(Eigen::VectorXd & Wsigma, double & kappa, int Nx)
00844     {
00845         int Nsigma = 2*Nx + 1;
00846
00847         Wsigma.resize(Nsigma,1);
00848
00849         kappa = (3 > Nx-3) ? 3 : Nx-3;
00850         Wsigma(0) = kappa/(Nx+kappa);
00851         for (int i = 1; i < Wsigma.rows(); ++i) {
00852             Wsigma(i) = 1/(2.0*(Nx+kappa));
00853         }
00854         Wsigma = Wsigma / Wsigma.sum(); // Garante soma 1.
00855     }
00856 public:
00857     // Override virtual members:
00858     virtual ~UnscentedKalmanFilter() {std::cout << "\nUnscentedKalmanFilter derived class
destroyed\n";}
00859
00860     void Reset(StateSpaceNonlinearModelBase& nonlinear_model) override
00861     {
00862         BayesianFilter::Reset(nonlinear_model.X0, nonlinear_model.P0, nonlinear_model.Nx,
nonlinear_model.Ny);
00863     }
00864
00865     Result Prediction(Eigen::VectorXd & U, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
00866     {
00867         Result res = BayesianFiltering::ResultFormat(true,"");
00868
00869         // Prediction
00870         // The function x(k) = f(x(k-1),u(k),w(k),t(k)) is converted to x(k) = f(xa(k),u(k),t(k))
with xa(k) = [x(k-1); w(k)]
00871         int Nx = nonlinear_model.Nx;

```

```

00889         int Nw = nonlinear_model.Nw;
00890         int DimX = Nx + Nw;
00891         int DimY = Nx;
00892
00893         Eigen::VectorXd Wsigma;
00894         double kappa;
00895         InitializeWeights(Wsigma, kappa, DimX);
00896         int Nsigma = Wsigma.rows();
00897
00898         // Generate augmented Xa and Pa
00899         Eigen::VectorXd Xa(DimX);
00900         Xa << X, Eigen::VectorXd::Zero(Nw);
00901
00902         Eigen::MatrixXd Pa(DimX,DimX);
00903         Pa << P, Eigen::MatrixXd::Zero(Nx,Nw),
00904             Eigen::MatrixXd::Zero(Nw, Nx), nonlinear_model.Q;
00905
00906         // Generate sigma points:
00907         Eigen::MatrixXd Xsigma(DimX,Nsigma);
00908         Eigen::MatrixXd Ysigma(DimY,Nsigma);
00909
00910         Eigen::LLT<Eigen::MatrixXd> llt((DimX+kappa)*Pa);
00911         Eigen::MatrixXd L = llt.matrixL();
00912
00913         Eigen::VectorXd Xi(Nx);
00914         Eigen::VectorXd Wi(Nw);
00915
00916         for (int i = 0; i < Nsigma; ++i) {
00917             if (i==0){
00918                 Xsigma.col(i) = Xa;
00919             }
00920             else if (i <= DimX){
00921                 Xsigma.col(i) = Xa + L.col(i-1);
00922             }
00923             else{
00924                 Xsigma.col(i) = Xa - L.col(i-1-DimX);
00925             }
00926             Xi = Xsigma.block(0 , i, Nx, 1);
00927             Wi = Xsigma.block(Nx, i, Nw, 1);
00928             Result res = nonlinear_model.ProcessModel(Xi,U,t,Wi);
00929             if(!res.first) return res;
00930             Ysigma.col(i) = Xi;
00931         }
00932         // Compute prediction
00933         X = Eigen::VectorXd::Zero(Nx);
00934         for (int i = 0; i < Nsigma; ++i) {
00935             X += Wsigma(i)*Ysigma.col(i);
00936         }
00937         P = Eigen::MatrixXd::Zero(Nx,Nx);
00938         for (int i = 0; i < Nsigma; ++i) {
00939             P += Wsigma(i)*(Ysigma.col(i)-X)*(Ysigma.col(i)-X).transpose();
00940         }
00941
00942         return res;
00943     }
00944
00951     Result Correction(Eigen::VectorXd & Y, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override
00952     {
00953         Result res = BayesianFiltering::ResultFormat(true,"");
00954
00955         // Correction
00956         // The function y(k) = h(x(k),v(k)) is converted to x(k) = h(xa(k)) with xa(k) = [x(k);
v(k)]
00957         int Nx = nonlinear_model.Nx;
00958         int Ny = nonlinear_model.Ny;
00959         int Nv = nonlinear_model.Nv;
00960         int DimX = Nx + Nv;
00961         int DimY = Ny;
00962
00963         Eigen::VectorXd Wsigma;
00964         double kappa;
00965         InitializeWeights(Wsigma, kappa, DimX);
00966         int Nsigma = Wsigma.rows();
00967
00968         // Generate augmented Xa and Pa
00969         Eigen::VectorXd Xa(DimX);
00970         Xa << X, Eigen::VectorXd::Zero(Nv);
00971
00972         Eigen::MatrixXd Pa(DimX,DimX);
00973         Pa << P, Eigen::MatrixXd::Zero(Nx,Nv),
00974             Eigen::MatrixXd::Zero(Nv,Nx), nonlinear_model.R;
00975
00976         // Generate sigma points:
00977         Eigen::MatrixXd Xsigma(DimX,Nsigma); Xsigma = Eigen::MatrixXd::Zero(DimX,Nsigma);
00978         Eigen::MatrixXd Ysigma(DimY,Nsigma); Ysigma = Eigen::MatrixXd::Zero(DimY,Nsigma);
00979

```

```

00980         Eigen::MatrixXd L;
00981         {
00982             Eigen::LLT<Eigen::MatrixXd> llt((DimX+kappa)*Pa);
00983             L = llt.matrixL();
00984         }
00985
00986         Eigen::VectorXd Xi(Nx);
00987         Eigen::VectorXd Vi(Nv);
00988         Eigen::VectorXd Yi(Ny);
00989         for (int i = 0; i < Nsigma; ++i) {
00990             if (i==0){
00991                 Xsigma.col(i) = Xa;
00992             }
00993             else if (i <= DimX){
00994                 Xsigma.col(i) = Xa + L.col(i-1);
00995             }
00996             else{
00997                 Xsigma.col(i) = Xa - L.col(i-1-DimX);
00998             }
00999             Xi = Xsigma.block(0 , i, Nx, 1);
01000             Vi = Xsigma.block(Nx, i, Nv, 1);
01001             res = nonlinear_model.MeasurementModel(Yi,Xi,t,Vi); if(!res.first) return res;
01002             Ysigma.col(i) = Yi;
01003         }
01004         // Compute correction
01005         Y_predicted = Eigen::VectorXd::Zero(Ny);
01006         for (int i = 0; i < Nsigma; ++i) {
01007             Y_predicted += Wsigma(i)*Ysigma.col(i);
01008         }
01009         Eigen::MatrixXd Pyy = Eigen::MatrixXd::Zero(Nx,Nx);
01010         Eigen::MatrixXd Pxy = Eigen::MatrixXd::Zero(Nx,Ny);
01011         for (int i = 0; i < Nsigma; ++i) {
01012             Pyy +=
01013             Wsigma(i)*(Ysigma.col(i)-Y_predicted)*((Ysigma.col(i)-Y_predicted).transpose());
01014             Pxy += Wsigma(i)*(Xsigma.block(0 , i, Nx,
01015             1)-X)*(Ysigma.col(i)-Y_predicted).transpose());
01016         }
01017         V_predicted = Y - Y_predicted;
01018         S_predicted = Pyy;
01019         d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
01020
01021         Eigen::MatrixXd G = Pxy * Pyy.inverse();
01022         X = X + G * V_predicted;
01023         P = P - G*Pyy*G.transpose();
01024
01025         // Compute corrected output, innovation and d^2:
01026         Xa << X, Eigen::VectorXd::Zero(Nv);
01027         Pa << P, Eigen::MatrixXd::Zero(Nx,Nv),
01028             Eigen::MatrixXd::Zero(Nv,Nx), nonlinear_model.R;
01029
01030         {
01031             Eigen::LLT<Eigen::MatrixXd> llt((DimX+kappa)*Pa);
01032             L = llt.matrixL();
01033         }
01034         for (int i = 0; i < Nsigma; ++i) {
01035             if (i==0){
01036                 Xsigma.col(i) = Xa;
01037             }
01038             else if (i <= DimX){
01039                 Xsigma.col(i) = Xa + L.col(i-1);
01040             }
01041             else{
01042                 Xsigma.col(i) = Xa - L.col(i-1-DimX);
01043             }
01044             Xi = Xsigma.block(0 , i, Nx, 1);
01045             Vi = Xsigma.block(Nx, i, Nv, 1);
01046             res = nonlinear_model.MeasurementModel(Yi,Xi,t,Vi); if(!res.first) return res;
01047             Ysigma.col(i) = Yi;
01048         }
01049         Y_corrected = Eigen::VectorXd::Zero(Ny);
01050         for (int i = 0; i < Nsigma; ++i) {
01051             Y_corrected += Wsigma(i)*Ysigma.col(i);
01052         }
01053         Pyy = Eigen::MatrixXd::Zero(Nx,Nx);
01054         for (int i = 0; i < Nsigma; ++i) {
01055             Pyy +=
01056             Wsigma(i)*(Ysigma.col(i)-Y_corrected)*((Ysigma.col(i)-Y_corrected).transpose());
01057             V_corrected = Y - Y_corrected;
01058             S_corrected = Pyy;
01059             d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
01060
01061             return res;
01062         }
01063     };

```

```

01070     class SamplingImportanceResamplingFilter : public NonlinearBayesianFilter
01071     {
01072     private:
01076         void ResetWeights()
01077         {
01078             long int Nparticles = Wparticles.rows();
01079             Wparticles.setConstant(1.0/((double) (Nparticles)));
01080         }
01081
01082     public:
01086         enum class EstimateType
01087         {
01088             MaximumAPosteriori,
01089             ConditionalMean,
01090         };
01091
01095         static inline const char* EnumToString(EstimateType x)
01096         {
01097             const std::map<EstimateType, const char*> Strings {
01098                 { EstimateType::MaximumAPosteriori, "Maximum a Posteriori estimate" },
01099                 { EstimateType::ConditionalMean, "Conditional Mean estimate" },
01100             };
01101             auto it = Strings.find(x);
01102             return it == Strings.end() ? "Out of range" : it->second;
01103         }
01104
01105         // Specific data
01106         EstimateType estimate_type;
01107         long int Nparticles;
01108         Eigen::MatrixXd Xparticles;
01109         Eigen::MatrixXd Yparticles;
01110         Eigen::VectorXd Wparticles;
01111         double ESS;
01112
01113         // Override virtual members:
01114         virtual ~SamplingImportanceResamplingFilter() {std::cout <<
"\nSamplingImportanceResamplingFilter derived class destroyed\n";}
01115
01122         void Reset(StateSpaceNonlinearModelBase& nonlinear_model, long int _Nparticles, EstimateType
_estimate_type)
01123         {
01124             BayesianFilter::Reset(nonlinear_model.X0, nonlinear_model.P0, nonlinear_model.Nx,
nonlinear_model.Ny);
01125             Nparticles = _Nparticles;
01126             estimate_type = _estimate_type;
01127             Xparticles.resize(nonlinear_model.Nx, Nparticles);
01128             Yparticles.resize(nonlinear_model.Ny, Nparticles);
01129             Wparticles.resize(Nparticles);
01130             ResetWeights();
01131             ESS = Nparticles;
01132         }
01133
01138         void Reset(StateSpaceNonlinearModelBase& nonlinear_model) override
01139         {
01140             Reset(nonlinear_model, 1000, EstimateType::ConditionalMean); // default number of
particles
01141         }
01142
01146         Result NormalizeWeights()
01147         {
01148             if (Wparticles.sum() > 0) {
01149                 double c = 1.0/Wparticles.sum();
01150                 Wparticles *= c;
01151             }
01152             else {
01153                 return ResultFormat(false, "SIR filter diverged: sum of weights = %f",
Wparticles.sum());
01154             }
01155             return BayesianFiltering::ResultFormat(true, "");
01156         }
01157
01161         Result ComputeESS()
01162         {
01163             double sum = 0;
01164
01165             for (long int i = 0; i < Wparticles.rows(); ++i) {
01166                 sum += Wparticles(i)*Wparticles(i);
01167             }
01168             if (sum <= 0.0) {
01169                 return ResultFormat(false, "SIR filter diverged: sum of square weights = %f", sum);
01170             }
01171             ESS = 1/sum;
01172             return BayesianFiltering::ResultFormat(true, "");
01173         }
01174
01181         Result Prediction(Eigen::VectorXd & U, double t, StateSpaceNonlinearModelBase&
nonlinear_model) override

```

```

01182     {
01183         Result res = BayesianFiltering::ResultFormat(true, "");
01184
01185         // Prediction
01186         // Generic nonlinear model  $x(k) = f(x(k-1), u(k), w(k), t(k))$ 
01187         int Nx = nonlinear_model.Nx;
01188         int Nw = nonlinear_model.Nw;
01189         long int Nparticles = Xparticles.cols();
01190
01191         ResetWeights();
01192
01193         Eigen::VectorXd Xi(Nx);
01194         Eigen::VectorXd Wi(Nw);
01195         for (long int i = 0; i < Nparticles; ++i) {
01196             Xi = Xparticles.col(i);
01197             res = MultivariateZeroMeanGaussianSampler(Wi, nonlinear_model.Q);
01198             if(!res.first) return res;
01199             res = nonlinear_model.ProcessModel(Xi, U, t, Wi);
01200             if(!res.first) return res;
01201             Xparticles.col(i) = Xi;
01202         }
01203
01204         // Compute prediction: assuming equal weights, Conditional Mean and Maximum a Posteriori
01205         // are the same.
01206         X = Eigen::VectorXd::Zero(Nx);
01207         for (long int i = 0; i < Nparticles; ++i) {
01208             X += Wparticles(i)*Xparticles.col(i);
01209         }
01210         P = Eigen::MatrixXd::Zero(Nx, Nx);
01211         for (long int i = 0; i < Nparticles; ++i) {
01212             P += Wparticles(i)*(Xparticles.col(i)-X)*(Xparticles.col(i)-X).transpose();
01213         }
01214
01215         // Particle Weights
01216         // No need to update;
01217
01218         res = ComputeESS(); if(!res.first) return res;
01219
01220         return res;
01221     }
01222
01223     Result Correction(Eigen::VectorXd & Y, double t, StateSpaceNonlinearModelBase&
01224     nonlinear_model) override
01225     {
01226         Result res = BayesianFiltering::ResultFormat(true, "");
01227
01228         // Correction
01229         // Generic nonlinear model  $y(k) = h(x(k), v(k))$ 
01230         int Nx = nonlinear_model.Nx;
01231         int Ny = nonlinear_model.Ny;
01232         int Nv = nonlinear_model.Nv;
01233         long int Nparticles = Wparticles.rows();
01234
01235         // Update weights
01236         // %%% Observar que os pesos q das particulas não são passadas como argumento. De fato,
01237         // assume-se que estes pesos
01238         // %%% são todos iguais a 1/Ns, decorrente da ultima execução do estimador. Assim, o
01239         // calculo da ponderação
01240         // %%%  $q(k) = q(k-1)*p(z(k)|x(k))$  seguido da normalização  $q(k) = q(k)/\sum(q)$  dá na mesma
01241         // coisa do que fazer
01242         // %%%  $q(k) = p(z(k)|x(k))$  seguido de  $q(k) = q(k)/\sum(q)$ , como está implementado.
01243         Eigen::VectorXd Xi(Nx);
01244         Eigen::VectorXd Vi(Nv);
01245         Eigen::VectorXd Yi(Ny);
01246         Y_predicted = Eigen::VectorXd::Zero(Ny); // Computer with weights equal, leading to the
01247         // mean
01248         for (long int i = 0; i < Nparticles; ++i) {
01249             Xi = Xparticles.col(i);
01250             res = MultivariateZeroMeanGaussianSampler(Vi, nonlinear_model.R); if(!res.first)
01251             return res;
01252             res = nonlinear_model.MeasurementModel(Yi, Xi, t, Vi); if(!res.first) return res;
01253             Y_predicted += Yi; // Simple mean
01254             Wparticles(i) = exp((-0.5)*(Y-Yi).transpose()*(nonlinear_model.R.inverse())*(Y-Yi));
01255
01256             Yparticles.col(i) = Yi;
01257         }
01258         Y_predicted /= ((double) (Nparticles));
01259         res = NormalizeWeights(); if(!res.first) return res;
01260
01261         res = ComputeESS(); if(!res.first) return res;
01262
01263         S_predicted = Eigen::MatrixXd::Zero(Ny, Ny);
01264         for (int i = 0; i < Nparticles; ++i) {
01265             S_predicted +=
01266             (Yparticles.col(i)-Y_predicted)*(Yparticles.col(i)-Y_predicted).transpose();
01267         }
01268         S_predicted /= ((double) (Nparticles));

```

```

01266     V_predicted = Y - Y_predicted;
01267     d2_predicted = (V_predicted.transpose() * S_predicted.inverse() * V_predicted)(0, 0);
01268
01269     // Resampling
01270     Eigen::VectorXd WparticlesCumSum(Nparticles);
01271     WparticlesCumSum(0) = Wparticles(0);
01272     for (long int i = 1; i < Nparticles; ++i) {
01273         WparticlesCumSum(i) = WparticlesCumSum(i-1) + Wparticles(i);
01274     }
01275     {
01276         long int i = 0;
01277         double u1 = UnivariateUniformSampler(0.0, 1.0)/((double) (Nparticles));
01278         for (long int j = 0; j < Nparticles; ++j) {
01279             double u = u1 + ((double) (j))/((double) (Nparticles));
01280             if (u > 1.0){ u = 1.0; }
01281             while (u > WparticlesCumSum(i)){
01282                 i = i + 1;
01283                 if (i > Nparticles-1){
01284                     i = Nparticles-1;
01285                     break;
01286                 }
01287             }
01288             Xparticles.col(j) = Xparticles.col(i);
01289         }
01290     }
01291     ResetWeights();
01292
01293     // Compute correction
01294     X = Eigen::VectorXd::Zero(Nx);
01295     switch (estimate_type){
01296     case EstimateType::ConditionalMean:
01297     {
01298         for (long int i = 0; i < Nparticles; ++i) {
01299             X += Wparticles(i)*Xparticles.col(i);
01300         }
01301     }
01302     break;
01303     case EstimateType::MaximumAPosteriori:
01304     {
01305         long int imax = 0;
01306         for (long int i = 1; i < Nparticles; ++i) {
01307             if (Wparticles(i) > Wparticles(imax)) imax = i;
01308         }
01309         X = Xparticles.col(imax);
01310     }
01311     break;
01312     }
01313
01314     P = Eigen::MatrixXd::Zero(Nx,Nx);
01315     for (long int i = 0; i < Nparticles; ++i) {
01316         P += Wparticles(i)*(Xparticles.col(i)-X)*(Xparticles.col(i)-X).transpose();
01317     }
01318
01319     // Compute corrected output, innovation and d^2:
01320     Y_corrected = Eigen::VectorXd::Zero(Ny); // Computer with weights equal, leading to the
mean
01321     for (long int i = 0; i < Nparticles; ++i) {
01322         Xi = Xparticles.col(i);
01323         res = MultivariateZeroMeanGaussianSampler(Vi, nonlinear_model.R); if(!res.first)
return res;
01324         res = nonlinear_model.MeasurementModel(Yi,Xi,t,Vi); if(!res.first) return res;
01325         Y_corrected += Yi; // Simple mean
01326         Yparticles.col(i) = Yi;
01327     }
01328     Y_corrected /= ((double) (Nparticles));
01329     S_corrected = Eigen::MatrixXd::Zero(Ny,Ny);
01330     for (int i = 0; i < Nparticles; ++i) {
01331         S_corrected +=
(Yparticles.col(i)-Y_corrected)*((Yparticles.col(i)-Y_corrected).transpose());
01332     }
01333     S_corrected /= ((double) (Nparticles));
01334     V_corrected = Y - Y_corrected;
01335     d2_corrected = (V_corrected.transpose() * S_corrected.inverse() * V_corrected)(0, 0);
01336
01337     return res;
01338 }
01339
01340 };
01341 }
01342
01343 #endif // BayesianFiltering_H

```

8.3 README.md File Reference

Index

- ~AdaptiveKalmanFilter
 - BayesianFiltering::AdaptiveKalmanFilter, [20](#)
- ~BayesianFilter
 - BayesianFiltering::BayesianFilter, [23](#)
- ~ExtendedKalmanFilter
 - BayesianFiltering::ExtendedKalmanFilter, [27](#)
- ~KalmanFilter
 - BayesianFiltering::KalmanFilter, [31](#)
- ~LinearBayesianFilter
 - BayesianFiltering::LinearBayesianFilter, [34](#)
- ~NonlinearBayesianFilter
 - BayesianFiltering::NonlinearBayesianFilter, [37](#)
- ~RobustSelfAdaptiveKalmanFilter
 - BayesianFiltering::RobustSelfAdaptiveKalmanFilter, [40](#)
- ~SamplingImportanceResamplingFilter
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- ~StateSpaceNonlinearModelBase
 - BayesianFiltering::StateSpaceNonlinearModelBase, [54](#)
- ~UnscentedKalmanFilter
 - BayesianFiltering::UnscentedKalmanFilter, [60](#)
- A
 - BayesianFiltering::StateSpaceLinearModel, [50](#)
- About this library, [1](#)
- alpha
 - BayesianFiltering::AdaptiveKalmanFilter, [21](#)
- B
 - BayesianFiltering::StateSpaceLinearModel, [50](#)
- BayesianFilter
 - BayesianFiltering::BayesianFilter, [23](#)
- BayesianFiltering, [13](#)
 - MultivariateGaussianSampler, [14](#)
 - MultivariateZeroMeanGaussianSampler, [14](#)
 - Result, [14](#)
 - ResultFormat, [15](#)
 - UnivariateGaussianSampler, [15](#)
 - UnivariateUniformSampler, [16](#)
- bayesianfiltering.hpp, [63](#)
 - PRINT_EIGEN_MATRIX, [65](#)
 - PRINT_EIGEN_VECTOR, [65](#)
 - PRINT_LINE_DEBUG, [65](#)
 - PrintEigenMatrixXd, [66](#)
 - PrintEigenVectorXd, [66](#)
- BayesianFiltering::AdaptiveKalmanFilter, [17](#)
 - ~AdaptiveKalmanFilter, [20](#)
 - alpha, [21](#)
 - beta, [21](#)
 - Correction, [20](#)
 - Prediction, [20](#)
 - Reset, [20, 21](#)
- BayesianFiltering::BayesianFilter, [22](#)
 - ~BayesianFilter, [23](#)
 - BayesianFilter, [23](#)
 - d2_corrected, [24](#)
 - d2_predicted, [24](#)
 - P, [24](#)
 - Reset, [23](#)
 - S_corrected, [24](#)
 - S_predicted, [24](#)
 - V_corrected, [24](#)
 - V_predicted, [24](#)
 - X, [25](#)
 - Y_corrected, [25](#)
 - Y_predicted, [25](#)
- BayesianFiltering::ExtendedKalmanFilter, [25](#)
 - ~ExtendedKalmanFilter, [27](#)
 - Correction, [27](#)
 - Prediction, [28](#)
 - Reset, [28](#)
- BayesianFiltering::KalmanFilter, [29](#)
 - ~KalmanFilter, [31](#)
 - Correction, [31](#)
 - Prediction, [31](#)
 - Reset, [32](#)
- BayesianFiltering::LinearBayesianFilter, [32](#)
 - ~LinearBayesianFilter, [34](#)
 - CheckModel, [34](#)
 - Correction, [34](#)
 - Prediction, [34](#)
 - Reset, [34](#)
- BayesianFiltering::NonlinearBayesianFilter, [35](#)
 - ~NonlinearBayesianFilter, [37](#)
 - Correction, [37](#)
 - Prediction, [37](#)
 - Reset, [37](#)
- BayesianFiltering::RobustSelfAdaptiveKalmanFilter, [38](#)
 - ~RobustSelfAdaptiveKalmanFilter, [40](#)
 - Correction, [40](#)
 - Prediction, [40](#)
 - Reset, [41](#)
- BayesianFiltering::SamplingImportanceResamplingFilter, [41](#)
 - ~SamplingImportanceResamplingFilter, [45](#)
 - ComputeESS, [45](#)
 - ConditionalMean, [45](#)

- Correction, [45](#)
- EnumToString, [45](#)
- ESS, [47](#)
- estimate_type, [47](#)
- EstimateType, [44](#)
- MaximumAPosteriori, [45](#)
- NormalizeWeights, [45](#)
- Nparticles, [47](#)
- Prediction, [46](#)
- Reset, [46](#)
- Wparticles, [47](#)
- Xparticles, [47](#)
- Yparticles, [47](#)
- BayesianFiltering::StateSpaceLinearModel, [48](#)
 - A, [50](#)
 - B, [50](#)
 - C, [50](#)
 - MeasurementModel, [49](#)
 - Nu, [50](#)
 - Nv, [51](#)
 - Nw, [51](#)
 - Nx, [51](#)
 - Ny, [51](#)
 - P0, [51](#)
 - ProcessModel, [49](#)
 - Q, [51](#)
 - R, [51](#)
 - SetDimensions, [50](#)
 - X0, [51](#)
- BayesianFiltering::StateSpaceNonlinearModelBase, [52](#)
 - ~StateSpaceNonlinearModelBase, [54](#)
 - dfdW, [56](#)
 - dfdX, [56](#)
 - dhdV, [56](#)
 - dhdX, [57](#)
 - MeasurementModel, [54](#)
 - Nu, [57](#)
 - Nv, [57](#)
 - Nw, [57](#)
 - Nx, [57](#)
 - Ny, [57](#)
 - P0, [57](#)
 - ProcessModel, [54](#)
 - Q, [57](#)
 - R, [58](#)
 - SetDimensions, [54](#)
 - UpdatedfdW, [55](#)
 - UpdatedfdX, [55](#)
 - UpdatedhdV, [56](#)
 - UpdatedhdX, [56](#)
 - X0, [58](#)
- BayesianFiltering::UnscentedKalmanFilter, [58](#)
 - ~UnscentedKalmanFilter, [60](#)
 - Correction, [61](#)
 - Prediction, [62](#)
 - Reset, [62](#)
- beta
 - BayesianFiltering::AdaptiveKalmanFilter, [21](#)
- C
 - BayesianFiltering::StateSpaceLinearModel, [50](#)
- CheckModel
 - BayesianFiltering::LinearBayesianFilter, [34](#)
- ComputeESS
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- ConditionalMean
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- Correction
 - BayesianFiltering::AdaptiveKalmanFilter, [20](#)
 - BayesianFiltering::ExtendedKalmanFilter, [27](#)
 - BayesianFiltering::KalmanFilter, [31](#)
 - BayesianFiltering::LinearBayesianFilter, [34](#)
 - BayesianFiltering::NonlinearBayesianFilter, [37](#)
 - BayesianFiltering::RobustSelfAdaptiveKalmanFilter, [40](#)
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
 - BayesianFiltering::UnscentedKalmanFilter, [61](#)
- d2_corrected
 - BayesianFiltering::BayesianFilter, [24](#)
- d2_predicted
 - BayesianFiltering::BayesianFilter, [24](#)
- dfdW
 - BayesianFiltering::StateSpaceNonlinearModelBase, [56](#)
- dfdX
 - BayesianFiltering::StateSpaceNonlinearModelBase, [56](#)
- dhdV
 - BayesianFiltering::StateSpaceNonlinearModelBase, [56](#)
- dhdX
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- EnumToString
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- ESS
 - BayesianFiltering::SamplingImportanceResamplingFilter, [47](#)
- estimate_type
 - BayesianFiltering::SamplingImportanceResamplingFilter, [47](#)
- EstimateType
 - BayesianFiltering::SamplingImportanceResamplingFilter, [44](#)
- MaximumAPosteriori
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- MeasurementModel
 - BayesianFiltering::StateSpaceLinearModel, [49](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [54](#)

- MultivariateGaussianSampler
 - BayesianFiltering, [14](#)
- MultivariateZeroMeanGaussianSampler
 - BayesianFiltering, [14](#)
- NormalizeWeights
 - BayesianFiltering::SamplingImportanceResamplingFilter, [45](#)
- Nparticles
 - BayesianFiltering::SamplingImportanceResamplingFilter, [47](#)
- Nu
 - BayesianFiltering::StateSpaceLinearModel, [50](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- Nv
 - BayesianFiltering::StateSpaceLinearModel, [51](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- Nw
 - BayesianFiltering::StateSpaceLinearModel, [51](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- Nx
 - BayesianFiltering::StateSpaceLinearModel, [51](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- Ny
 - BayesianFiltering::StateSpaceLinearModel, [51](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- P
 - BayesianFiltering::BayesianFilter, [24](#)
- P0
 - BayesianFiltering::StateSpaceLinearModel, [51](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- Prediction
 - BayesianFiltering::AdaptiveKalmanFilter, [20](#)
 - BayesianFiltering::ExtendedKalmanFilter, [28](#)
 - BayesianFiltering::KalmanFilter, [31](#)
 - BayesianFiltering::LinearBayesianFilter, [34](#)
 - BayesianFiltering::NonlinearBayesianFilter, [37](#)
 - BayesianFiltering::RobustSelfAdaptiveKalmanFilter, [40](#)
 - BayesianFiltering::SamplingImportanceResamplingFilter, [46](#)
 - BayesianFiltering::UnscentedKalmanFilter, [62](#)
- PRINT_EIGEN_MATRIX
 - bayesianfiltering.hpp, [65](#)
- PRINT_EIGEN_VECTOR
 - bayesianfiltering.hpp, [65](#)
- PRINT_LINE_DEBUG
 - bayesianfiltering.hpp, [65](#)
- PrintEigenMatrixXd
 - bayesianfiltering.hpp, [66](#)
- PrintEigenVectorXd
 - bayesianfiltering.hpp, [66](#)
- ProcessModel
 - BayesianFiltering::StateSpaceLinearModel, [49](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [54](#)
- Q
 - BayesianFiltering::StateSpaceLinearModel, [51](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [57](#)
- R
 - BayesianFiltering::StateSpaceLinearModel, [51](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [58](#)
- README.md, [78](#)
- Reset
 - BayesianFiltering::AdaptiveKalmanFilter, [20, 21](#)
 - BayesianFiltering::BayesianFilter, [23](#)
 - BayesianFiltering::ExtendedKalmanFilter, [28](#)
 - BayesianFiltering::KalmanFilter, [32](#)
 - BayesianFiltering::LinearBayesianFilter, [34](#)
 - BayesianFiltering::NonlinearBayesianFilter, [37](#)
 - BayesianFiltering::RobustSelfAdaptiveKalmanFilter, [41](#)
 - BayesianFiltering::SamplingImportanceResamplingFilter, [46](#)
 - BayesianFiltering::UnscentedKalmanFilter, [62](#)
- Result
 - BayesianFiltering, [14](#)
- ResultFormat
 - BayesianFiltering, [15](#)
- S_corrected
 - BayesianFiltering::BayesianFilter, [24](#)
- S_predicted
 - BayesianFiltering::BayesianFilter, [24](#)
- SetDimensions
 - BayesianFiltering::StateSpaceLinearModel, [50](#)
 - BayesianFiltering::StateSpaceNonlinearModelBase, [54](#)
- UnivariateGaussianSampler
 - BayesianFiltering, [15](#)
- UnivariateUniformSampler
 - BayesianFiltering, [16](#)
- UpdatedfdW
 - BayesianFiltering::StateSpaceNonlinearModelBase, [55](#)
- UpdatedfdX
 - BayesianFiltering::StateSpaceNonlinearModelBase, [55](#)
- UpdatedhdV
 - BayesianFiltering::StateSpaceNonlinearModelBase, [56](#)
- UpdatedhdX
 - BayesianFiltering::StateSpaceNonlinearModelBase, [56](#)
- V_corrected

BayesianFiltering::BayesianFilter, [24](#)

V_predicted
BayesianFiltering::BayesianFilter, [24](#)

Wparticles
BayesianFiltering::SamplingImportanceResamplingFilter,
[47](#)

X
BayesianFiltering::BayesianFilter, [25](#)

X0
BayesianFiltering::StateSpaceLinearModel, [51](#)
BayesianFiltering::StateSpaceNonlinearModelBase,
[58](#)

Xparticles
BayesianFiltering::SamplingImportanceResamplingFilter,
[47](#)

Y_corrected
BayesianFiltering::BayesianFilter, [25](#)

Y_predicted
BayesianFiltering::BayesianFilter, [25](#)

Yparticles
BayesianFiltering::SamplingImportanceResamplingFilter,
[47](#)